

Front-End JavaScript: Make Your Web Pages Come Alive

The full teaching manual for the five-day course. Timing, live-demo scripts, the beginner mistakes to pre-empt, lab facilitation, and checks for understanding — for all six modules.

6 modules · 22 lessons

~18.3 teaching hours · 30 h with labs & breaks

Instructor copy · not for students

CONTENTS

MOD 1 JavaScript Foundations

- L01 Welcome to JavaScript
- L02 Linking Scripts & the Console
- L03 Variables & Data Types
- L04 Operators & Expressions

MOD 2 Control Flow & Functions

- L05 Conditionals
- L06 Loops
- L07 Functions
- L08 Scope & Hoisting

MOD 3 Working with Data

- L09 Arrays
- L10 Objects
- L11 Array Methods
- L12 Strings & Template Literals

MOD 4 The DOM

- L13 Introduction to the DOM
- L14 Selecting & Modifying Elements
- L15 Events & Event Listeners
- L16 Creating & Removing Elements

MOD 5 Building Interactive Pages

- L17 Forms & Validation
- L18 Timers & Animation
- L19 Local Storage
- L20 Working with APIs & Fetch

MOD 6 Capstone & Next Steps

- L21 Building a Complete Project
- L22 Next Steps & Best Practices

Module 1 — JavaScript Foundations

How to teach the four lessons that take students who already know HTML & CSS to their first real JavaScript: the Console, a linked script file, variables and data types, and operators. This is the start of the course — get every DevTools window open and every `app.js` linked before you move on.

4 Lessons

~2.75 hrs teaching

Lessons L01-L04

Level Beginner

Maps to Day 1 AM

HOW TO USE THIS GUIDE

- Same rhythm as every module: **Frame** (why it matters) → **Teach** (live-code the concept) → **Lab** (students build) → **Check** (quiz + recap). Every lesson has a 5-question quiz and ends with 3–5 minutes for the lesson's **Learning Journal** prompt (in the lesson and the workbook). The minute timings in each teaching sequence add up to the lesson's length; adjust ±10 min to your group.
- Live-code everything.** Type into the Console and into `app.js` in front of the class — never paste finished code. Each lesson has a short **live-demo script**: type it exactly, in order, and say what you expect *before* you press Enter.
- Blue blocks are concepts; teal blocks are hands-on; amber blocks are watch-outs. In Module 1 the watch-outs are mostly "where did my output go?" problems — the Console, the file link, and JavaScript's type surprises.
- Quiz answers here are for you. The student workbook has a 3-question subset of each lesson quiz, without answers, labelled with the lesson's question numbers; the separate **Answer Key** covers every question with matching numbers for grading.

BEFORE YOU START — DAY 1 SETUP CHECK (10 MIN, BEFORE L01)

Students come from the HTML & CSS course, so they should already have **VS Code** with the **Live Server** extension. Confirm it in the first ten minutes: everyone opens any folder in VS Code, right-clicks an `.html` file → **Open with Live Server**, and sees it at `http://127.0.0.1:5500/...`. Anyone missing Live Server installs it from the Extensions panel (search "Live Server" by Ritwick Dey). Then have **everyone open DevTools and the Console** on that page, on their own OS — this is the most-used shortcut of the whole course:

BROWSER	WINDOWS / LINUX	MAC
Any browser	F12 → CONSOLE TAB	CMD+OPTION+I → CONSOLE TAB
Chrome / Edge	CTRL+SHIFT+J	CMD+OPTION+J
Firefox	CTRL+SHIFT+K	CMD+OPTION+K
Safari	—	ENABLE SETTINGS → ADVANCED → "SHOW FEATURES FOR WEB DEVELOPERS", THEN CMD+OPTION+C

Recommend Chrome or Edge for class so your screen matches theirs. If anyone is behind, pair them up — do not stop the room to fix one laptop.

MODULE AT A GLANCE

LESSON	TOPIC	TIME	STUDENTS LEAVE ABLE TO...
L01	Welcome to JavaScript	30m	Explain JS's role beside HTML/CSS and change a live page from the Console
L02	Linking Scripts & the Console	40m	Link an external <code>app.js</code> and read its console output
L03	Variables & Data Types	50m	Store values with <code>const</code> / <code>let</code> and check types with <code>typeof</code>
L04	Operators & Expressions	45m	Calculate, compare with <code>===</code> , and combine conditions

The module is one story: **see what JavaScript does → connect it to a page → store data → work with data**. From L02 on, the labs build **one cumulative project**, the **js-practice** folder: L02 creates it, L03 adds a user profile to **app.js**, L04 adds a tip calculator. Everything is console output in this module — putting results on the page comes in Module 4.

L01 Welcome to JavaScript

◆ OBJECTIVES

- ◆ Explain what JavaScript is and what role it plays alongside HTML and CSS
- ◆ Describe the difference between front-end and back-end JavaScript
- ◆ Understand how the browser executes JavaScript
- ◆ Identify real-world examples of JavaScript in action on web pages
- ◆ Outline what they'll build throughout this course

◆ TEACHING SEQUENCE

- 1 **Frame (3m).** "Your pages look great, but they don't *do* anything." Open a site the class uses and click a menu or type in a search box — that's JavaScript. Add the third layer to the house analogy they know: HTML is the skeleton, CSS the paint and furniture, **JavaScript the electricity, plumbing, and elevator.**
- 2 **The three layers (5m).** Structure / presentation / behavior. Sell the head start: "You already know two-thirds of this."
- 3 **JavaScript in action (3m).** Rapid-fire list from the room: dropdowns, live search, infinite scroll, form checks, a cart that remembers you. Land the aside: **JavaScript ≠ Java** — the name was a 1990s marketing decision.
- 4 **How browsers run it (4m).** Every browser has a built-in engine (V8, SpiderMonkey, JavaScriptCore) — nothing to install. Walk the load story: parse HTML → load CSS and JS → display → *wait for the user* → react. That last loop is why front-end JS is called **event-driven**.
- 5 **History & ECMAScript (2m).** Ten days in 1995; ES6 (2015) is where "modern JS" starts. ECMAScript = the rulebook; JavaScript = what browsers run.
- 6 **Front-end vs back-end + roadmap (3m).** This course is front-end (the browser); Node.js is the server side, for later. Preview the six modules.
- 7 **Lab — Spot JavaScript in the Wild (6m).** Console worksheet (below).
- 8 **Check + journal (4m).** Quiz poll, then the journal prompt.

◆ LIVE-DEMO SCRIPT

On the practice page (`labs/module-1/l01-welcome-to-javascript/starter/index.html`), open the Console and type each line, predicting out loud first:

```
document.title
document.title = "I changed this with JavaScript!"
document.body.style.backgroundColor = "lightyellow"
```

Point at the browser tab after line 2, then refresh: everything comes back. "You changed *your copy* of the page, never the site's files." Repeat on a real site the class knows for the wow.

◆ LAB — SPOT JAVASCRIPT IN THE WILD

No files to write. Students follow the guided console worksheet in `labs/module-1/l01-welcome-to-javascript/README.md` : run the three statements on the practice page, refresh, then repeat on a site they use and list three features they think JavaScript drives. The solution page replays the statements from a script (with the output mirrored on screen) if you want to show the result on the projector.

Circulate for: the Console tab (not Elements) selected; the cursor in the Console's input line at the bottom; straight quotes, not curly ones.

◆ WATCH-OUTS

- ◆ **"Allow pasting" block.** Chrome and Firefox refuse the first paste into the Console and ask you to type `allow pasting` . It's a real anti-scram feature, not an error. Tell students to *type* the lines — better practice anyway.
- ◆ **"I broke the website!"** Some students fear the change is permanent. Refresh proves it isn't — and that answers quiz Q4.
- ◆ **Curly quotes** copied from a document cause `SyntaxError: Invalid or unexpected token` . Retype the quotes.

- ◆ **Going too deep.** Don't explain the DOM, `document`, or the engine internals yet. Today they only need "JavaScript can read and change the page."

◆ CHECK FOR UNDERSTANDING — QUIZ (FROM THE LESSON) + JOURNAL

Answers: **Q1** C) Adding interactivity and behavior; **Q2** B) False; **Q3** C) Nothing extra — browsers have a built-in JavaScript engine; **Q4** A) The text in the browser tab changed, just for your copy of the page; **Q5** D) The official specification (rulebook) that JavaScript follows. Full set in the Answer Key. Then give 3–5 minutes for the lesson's **Learning Journal** prompt: *Pick a website you use every day and list three things on it that you now think are powered by JavaScript. For each one, explain why HTML and CSS alone couldn't do it. Name one feature you'd love to add to your own site — and one thing about how JavaScript works that still feels fuzzy.* Students who prefer paper can use the workbook's Journal box; the online box saves only on that device.

OBJECTIVES

- Add JavaScript to an HTML page using the `<script>` tag
- Tell inline scripts from external script files — and why external wins
- Use `console.log()` and the other console methods
- Open and use the DevTools Console for testing and debugging
- Explain why script placement matters (head vs. body, `defer`)

TEACHING SEQUENCE

- 1 **Frame (2m)**. "Last lesson you typed JavaScript into the Console. Now we make it part of the page, so it runs every time the page loads."
- 2 **Inline script (4m)**. Add a `<script>console.log("Hello from JavaScript!")</script>` before `</body>`. The message is in the Console, *not on the page* — say that twice.
- 3 **External file (6m)**. Move the code to `app.js` and link it with `<script src="app.js"></script>`. Compare it to linking CSS with `<link>`: separation of concerns, reuse, caching, errors that name a file and line. Then show the trap: code *between* the tags of a `src` script is ignored.
- 4 **Placement & `defer` (7m)**. Use the diagram below and the demo script: a script in the `<head>` runs before the body exists. Two fixes: before `</body>` (our course default) or `defer` in the head. Both are correct.
- 5 **The Console as a playground (4m)**. Shortcuts recap, then type `2 + 2`, `"Hello" + " " + "World"`, `Math.random()`.
- 6 **Console methods (4m)**. `log`, `info`, `warn` (yellow), `error` (red), `table`, `group`, `time`. Pitch `console.log()` as their #1 debugging tool.
- 7 **Comments (3m)**. `//` and `/* */`. Comment the *why*, not the *what* — quiz Q5 tests exactly this.
- 8 **Lab — Your First JS Project (6m)**. Build `js-practice`.
- 9 **Check + journal (4m)**.

DIAGRAM — WHEN DOES APP.JS RUN?

TIME →

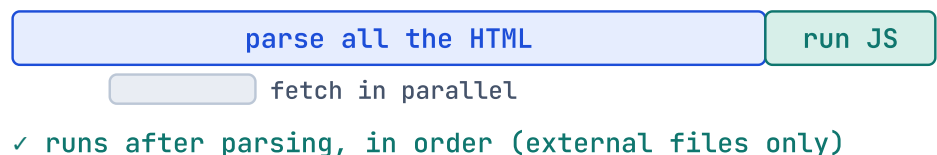
`<head>`
no `defer`



before
`</body>`



`<head>`
+ `defer`



Where the `<script>` tag sits decides when `app.js` runs.

LIVE-DEMO SCRIPT — PLACEMENT

Put this in `app.js` and move the `<script>` tag between the three positions. No DOM skills needed — `document.body` is either there or it isn't:

```
// app.js
console.log("Is the body ready?", document.body);
```

In the `<head>`: `null`. Before `</body>`: the `<body>` element. Back in the head with `defer`: the element again. That one line makes the whole placement rule concrete.

◆ LAB — YOUR FIRST JS PROJECT

Students create a `js-practice` folder with `index.html` and `app.js`, link the script before `</body>`, and try every console method from the lesson. Starter in `labs/module-1/l02-linking-scripts/starter/`; solution in `labs/module-1/l02-linking-scripts/solution/`. This folder carries through L03 and L04 — tell them to keep it.

Circulate for: both files in the *same* folder; `src="app.js"` spelled exactly like the file; the page opened with Live Server; the Console open. Expected output: log, yellow warning, red "not a real one!" error, info, `5 + 3 = 8`, `10 * 4 = 40`, and a three-row table.

◆ WATCH-OUTS

- ◆ **Nothing in the Console.** Check in order: file saved? `src` matches the file name, including case (`App.js` $\neq$ `app.js` on Linux servers)? Both files in the same folder? The Network tab shows a 404 for a wrong path — there is no other error.
- ◆ **Code inside a `src` script** is silently ignored. Use a second `<script>` tag.
- ◆ **Script in the `<head>` without `defer`** — fine for console-only code today, but the source of "can't find my element" bugs in Module 4. Build the before-`</body>` habit now.
- ◆ **The red line scares people.** `console.error("...not a real one!")` is the demo. A *real* error starts with `Uncaught` and names a file and line — teach them to click that link.
- ◆ **The grey `undefined`** after typing `console.log("hi")` in the Console is the return value of `console.log`, not a problem.
- ◆ **Filtered console.** If warnings or info are missing, the log-level dropdown ("Default levels") or the filter box is hiding them.

◆ CHECK FOR UNDERSTANDING — QUIZ (FROM THE LESSON) + JOURNAL

Answers: Q1 B) Just before the closing `</body>` tag; Q2 A) Prints a message to the browser's Developer Tools Console; Q3 C) Downloads the script in parallel and runs it after HTML parsing completes; Q4 D) `app.js` runs, but the `console.log()` inside the tags is ignored; Q5 A) `// Start at 3 lives so new players aren't punished too early`. Full set in the Answer Key. Journal prompt: *Imagine a script in the `<head>` (without `defer`) tries to find your page's `<h1>`. Explain what goes wrong and two ways to fix it. Which will you use, and why? What's still unclear about when scripts run?*

◆ OBJECTIVES

- ◆ Declare variables using `let`, `const`, and `var` — and explain the difference
- ◆ Identify the primitive types: string, number, boolean, null, undefined, symbol, bigint
- ◆ Use `typeof` to check a value's type
- ◆ Follow naming rules and conventions (camelCase, `UPPER_SNAKE_CASE`, `is` / `has` booleans)

◆ TEACHING SEQUENCE

- 1 **Frame (2m).** Programs remember things: a name, a score, a cart, "is logged in." Variables are how; types say what kind of thing.
- 2 **Variables as labeled boxes (5m).** `let username = "Ray";` — the label is the name, the value is inside. `=` is **assignment** ("store this"), not "equals." Reassign `mood` from `"happy"` to `"excited"`.
- 3 **`const` / `let` / `var` (8m).** Reassign a `const` live and read the `TypeError` aloud. Rule of thumb: `const` by default, `let` when it changes, never `var` in new code.
- 4 **Naming (4m).** Hard rules (letters, digits, `_`, `$`; no leading digit; case-sensitive; no reserved words) vs conventions. Poll the class on `2ndPlace`, `user-name`, `$totalScore` — that's quiz Q5.
- 5 **Data types (9m).** String (three kinds of quotes, `.length`, template literals with `${...}`), number (one type for integers and decimals; `NaN`, `Infinity`), boolean, `null` (deliberately empty) vs `undefined` (not assigned yet). Symbol and bigint: name them, skip them.
- 6 **`typeof` (4m).** Run it on each type; save `typeof null` → `"object"` for last and tell the 1995 story. Check null with `=== null`.
- 7 **Type conversion (6m).** Explicit: `Number("42")`, `String(19.99)`, `Boolean("")`. Implicit: `"5" + 3` vs `"5" - 3` — the biggest gotcha of the day.
- 8 **Lab — Build a User Profile (8m).**
- 9 **Check + journal (4m).**

◆ LIVE-DEMO SCRIPT

```
const SITE_NAME = "My Website";
let count = 0;
count = 1;           // fine - let can change
SITE_NAME = "Other"; // TypeError: Assignment to constant variable.

let futureValue;
console.log(futureValue); // undefined
console.log(typeof null); // "object" - the famous bug
console.log("5" + 3);     // "53"
console.log("5" - 3);     // 2
```

Pause before each `console.log` and take predictions from the room. Wrong predictions are the point.

◆ LAB — BUILD A USER PROFILE

In `js-practice/app.js`, students declare seven profile variables (`firstName` ... `nickname = null`), log each with its `typeof`, build a template-literal greeting, update the `let` values, then try reassigning a `const`. Starter (L02 checkpoint + the lesson's TODO block) in `labs/module-1/l03-variables-data-types/starter/`; solution in `labs/module-1/l03-variables-data-types/solution/`.

Circulate for: `const` on `firstName/lastName/email` and `let` on the rest; backticks (not quotes) around template literals; the `const` reassignment left *commented out* afterwards, because an uncaught `TypeError` stops the rest of the file.

◆ WATCH-OUTS

- ◆ **Floating-point output.** `149.99 - 29.99` prints `120.00000000000001`. Not their bug: decimals are stored in binary. Format money with `.toFixed(2)` — the lesson solution does, and L04 relies on it.

- ◆ **"Identifier has already been declared."** Students paste several lesson snippets into one `app.js` and declare `price` or `name` twice. That's a `SyntaxError`, and *nothing* in the file runs — not even the lines above it. (The Console lets you re-run `let` lines; a file doesn't. Say this out loud, or the difference will confuse them.)
- ◆ **`const` with objects and arrays.** Someone will find that `const colors = ["red"]; colors.push("blue");` works. `const` locks the *variable*, not the contents — you can't point it at a new array, but you can change what's inside. Acknowledge it, then park it for Module 3.
- ◆ **Quotes vs backticks.** `"Hello ${name}"` prints the `${name}` literally. Only backticks interpolate.
- ◆ **`=` read as "equals."** Keep saying "gets" or "stores": `age = 31` is "age gets 31." Comparison is next lesson.

◆ CHECK FOR UNDERSTANDING — QUIZ (FROM THE LESSON) + JOURNAL

Answers: Q1 C) `const`; Q2 B) "object"; Q3 B) "53"; Q4 A) undefined; Q5 D) `$totalScore`. Full set in the Answer Key. Journal prompt: *Think about your own website from the HTML & CSS course. List five pieces of information it could store in variables. For each, choose a data type and `const` or `let`, and explain why. Then describe `null` vs `undefined` in your own words. Which part still feels shaky?*

◆ OBJECTIVES

- ◆ Use arithmetic operators, including `%` (remainder) and `**`
- ◆ Apply assignment shorthand (`+=`, `-=`, `*=` ...)
- ◆ Compare values and explain the critical difference between `==` and `===`
- ◆ Combine conditions with `&&`, `||`, `!`; know `true` and `false`
- ◆ Predict how an expression evaluates using operator precedence

◆ TEACHING SEQUENCE

- 1 **Frame (2m).** Variables store data; operators *do* something with it. An **expression** is any code that works out to a value — `2 + 3` is `5`.
- 2 **Arithmetic (5m).** The table from the lesson, then `%`: `10 % 3` is the **remainder**, 1. Even/odd with `n % 2`. `count++` / `count--`.
- 3 **Assignment shorthand (3m).** `score += 25` means `score = score + 25`. Trace a short chain on the board (that's quiz Q4).
- 4 **Comparison — `==` vs `===` (8m).** The heart of the lesson; see the demo script. Rule: **always `===` and `!==`**.
- 5 **Logical operators (5m).** `age >= 21 && hasID`, `isWeekend || isHoliday`, `!isLoggedIn`. Short-circuiting in one sentence, and `" " || "Guest"` → `"Guest"` as a preview.
- 6 **Truthy & falsy (4m).** The falsy list: `false`, `0`, `-0`, `0n`, `""`, `null`, `undefined`, `NaN`. Everything else is `true` — including `"0"`, `[]`, and `{}`.
- 7 **String `+` & precedence (4m).** `+` joins strings; prefer template literals. `2 + 3 * 4` is 14, not 20. "Use parentheses generously."
- 8 **Lab — Tip Calculator (10m).**
- 9 **Check + journal (4m).**

◆ LIVE-DEMO SCRIPT — THE EQUALITY LADDER

```
5 === 5           // true
5 === "5"        // false - different types
5 == "5"         // true - == converts first
"" == 0          // true (!)
"0" == 0         // true
"" == "0"        // false - so == isn't even consistent
null == undefined // true
```

Type these in the Console and let the class vote before each Enter. By line 6 the room will argue — then retype lines 4–6 with `===` and they all come out `false`. That settles the rule.

◆ LAB — TIP CALCULATOR

Students add a console tip calculator to `js-practice/app.js`: three `const` inputs, three calculations, four template-literal lines, and the `bonus isExpensive` comparison. Starter (L03 checkpoint + the lesson's TODO block) in `labs/module-1/l04-operators-expressions/starter/`; solution in `labs/module-1/l04-operators-expressions/solution/`.

Expected output: `Bill: $85.50 · Tip (20%): $17.10 · Total: $102.60 · Per person: $34.20 · Expensive? true`. Anyone showing `34.199999999999996` has forgotten `.toFixed(2)` — send them to the lesson's hint.

◆ WATCH-OUTS

- ◆ **Floating-point results.** `102.6 / 3` is `34.199999999999996`. It's normal; format for display with `.toFixed(2)`. Note that `.toFixed()` returns a `string` — format at the end, don't do more math on it.
- ◆ **`%` is not "percent."** Beginners write `billAmount * 20%`. `%` is remainder; a percentage is `* tipPercentage / 100`.
- ◆ **The ``${...}`` pattern.** In ``Bill: ${billAmount.toFixed(2)}`` the first `$` is a literal dollar sign and `${` starts the placeholder. Students often drop one.

- ◆ `==` "works" in their tests. `5 == 5` is true either way, so they think it doesn't matter. Show a mixed-type case every time.
- ◆ `const` for results. The lesson declares `tipAmount` etc. with `const` because they're calculated once. If a student uses `let`, that runs fine — just ask "does this value ever change?"

◆ CHECK FOR UNDERSTANDING – QUIZ (FROM THE LESSON) + JOURNAL

Answers: Q1 B) 1; Q2 A) false; Q3 B) true; Q4 C) 30; Q5 D) `"0"`. Full set in the Answer Key. Journal prompt: *Before running them, predict what `"5" + 3`, `"5" - 3`, `5 == "5"`, and `5 === "5"` give, and why. Check in the console. Which surprised you? Explain why the course says always use `===`. Think of a rule for your own site you could write with `&&` or `||` — what's still confusing about combining conditions?*

CLOSING THE MODULE

MODULE 1 WRAP-UP (5 MIN)

Everyone should now have a working `js-practice` folder whose console prints a profile and a tip calculation. Have one or two students read their tip-calculator output aloud. Recap the arc: **what JS does → link a script → store data → work with data**. Preview Module 2 (this afternoon): "Your code can calculate and compare. Next it makes decisions with `if / else`, repeats work with loops, and packages logic into functions."

Module 2 — Control Flow & Functions

How to teach the four lessons that take students from straight-line scripts to programs that decide, repeat, and organize themselves into functions. Timing, live-demo scripts, the misconceptions to pre-empt, and how to run each lab.

4 Lessons

~3.3 hrs teaching

Lessons L05-L08

Level Beginner → Intermediate

Maps to Day 1 PM (L05) → Day 2 AM

HOW TO USE THIS GUIDE

- Each lesson has the same rhythm: **Frame** (why it matters) → **Teach** (live-code the concept) → **Lab** (students build) → **Check** (quiz + recap) → **Journal**. The minute timings in each teaching sequence add up to the lesson's length; adjust ±10 min to your group, and let labs run into the day's lab block when you have one.
- Live-code everything.** Type the JavaScript in front of the class, save, and read the Console out loud. Never paste finished code. Beginners learn most from watching you hit an error, read the message, and fix it.
- Predict before you run.** Before every demo run, ask the room "what will this print?" and take two or three guesses. This whole module is about predicting program flow, and the habit matters more than any single syntax rule.
- Blue blocks are concepts; teal blocks are hands-on; amber blocks are watch-outs — the errors that eat class time if you don't pre-empt them.
- Quiz answers here are for you. The student workbook has a 3-question subset of each lesson quiz, without answers, labelled with the lesson's question numbers; the separate **Answer Key** covers every question with matching numbers for grading.

BEFORE YOU START THE MODULE

Students should have VS Code with Live Server, a browser with DevTools open on the **Console** tab, and a working `index.html` + `script.js` from Module 1. Every lab in this module is a console program: the lab pages in `labs/module-2/` also print each result onto the page with a provided `show()` helper, so nobody is stuck if DevTools is fiddly. If anyone is behind, pair them up — do not stop the room to fix one setup.

MODULE AT A GLANCE

LESSON	TOPIC	TIME	STUDENTS LEAVE ABLE TO...
L05	Conditionals	45m	Branch with if / else if / else, switch, and the ternary
L06	Loops	50m	Repeat work with for, while, for...of and for...in
L07	Functions	55m	Write declarations, expressions and arrows that return values
L08	Scope & Hoisting	45m	Predict where a variable is visible; build a closure

Total 195 minutes of lesson time plus a 10-minute module wrap-up. L05 closes Day 1; L06–L08 fill Day 2 morning. The labs are **four separate console programs**, not one cumulative project: a grade calculator, a star rating + search filter, a mini utility library, and a scope puzzle + closure score tracker. L08 is the first *intermediate* lesson in the course, so slow down there.

L05 Conditionals

OBJECTIVES

- Write `if`, `else if`, and `else` statements to control program flow
- Use `switch` for multi-branch decisions on one value
- Write short value choices with the ternary operator (`? :`)
- Replace deep nesting with guard clauses; use truthy/falsy values in conditions safely

TEACHING SEQUENCE

- Frame (3m).** "So far every line runs, every time. Real pages make choices: is the user logged in? Is the field empty? Is it morning?" Sketch a diamond on the board: condition → yes path / no path.
- if / else + logical operators (6m).** Live-code the voting-age check, then combine with `&&` (age and ticket) and `||` (weekend or holiday). Always type the braces, even for one line.
- else if chains — order matters (6m).** Build the grade chain from `>= 90` down. Then deliberately put `>= 60` first and show every passing score becomes a D. "First true condition wins; the rest are skipped."
- switch (6m).** The day-of-week example: stacked cases for Tuesday–Thursday, `break`, `default`. Delete one `break` to show fall-through, then put it back.
- Ternary (3m).** `const result = score >= 60 ? "Pass" : "Fail";` Read it aloud as "is it? then this : otherwise that." One level only — no nesting.
- Guard clauses + truthy/falsy (5m).** Show the nested `processOrder` pyramid next to the flat guard-clause version. Then `if (username)` vs the long `!== ""` chain, and the `0` trap.
- Lab (10m).** Grade Calculator (below).
- Check (3m).** Quiz Q1–Q5 as a quick poll.
- Journal (3m).** The lesson's prompt: one decision a page they'd like to build has to make, written as a plain-English if / else if / else chain.

LIVE-DEMO SCRIPT

```
let score = 82;

if (score >= 90) {
  console.log("Grade: A");
} else if (score >= 80) {
  console.log("Grade: B");
} else {
  console.log("Keep going");
}

// Ask: "82 is also >= 70 - why don't we see C?"
```

Say: "JavaScript walks down the chain and stops at the first true condition." Change `score` to 95, 82, 40 and have the room call out the result before each save. Then type `let input = "5";` and a `switch` with `case 5:` — let them predict, then show it falls to `default`. That's quiz Q4 taught in advance.

LAB — GRADE CALCULATOR

Students set `studentName` and `score`, pick the letter with an `else if` chain, pick the message with a `switch`, decide pass/fail with a ternary, and print *"Alex scored 78 (C) — Solid effort, keep improving!"* and *"Status: Pass"*. Bonus: reject scores below 0 or above 100 first. Starter in `labs/module-2/l05-conditionals/starter/`; solution in `labs/module-2/l05-conditionals/solution/`.

Circulate for: the chain checking highest first; a `break` in every case; the letter stored in a variable *before* the `switch`. Make them test: 95, 60, 59, -5 and 105 — every branch plus both sides of the 60 boundary.

◆ WATCH-OUTS

- ◆ `switch` uses strict equality (`===`). A string `"5"` never matches `case 5:`, and there's no error — it just drops to `default`. Anything read from a page is a string; convert with `Number()` first.
- ◆ `if (quantity)` fails when 0 is valid. `0` is falsy, so a real zero looks "missing." Check explicitly (`quantity !== undefined`) when zero is a legitimate value.
- ◆ `=` inside a condition. `if (score = 90)` assigns and is always truthy. When "every score is an A," look for a single equals sign.
- ◆ Guard clauses before functions. The lesson's guard-clause example uses `function` and `return`, which aren't taught until L07. Tell students to read `return;` as "stop here and leave." In the lab, the guard is an `if / else` wrapper, not a `return`.
- ◆ Missing `break` — the message for B shows the message for C too. Fall-through is only intentional for stacked cases.

◆ CHECK FOR UNDERSTANDING

Run quiz Q1–Q5 as a quick poll. Key answers: Q1 `x = 5` prints **B** (first true branch wins); Q2 missing `break` → falls through to the next case; Q3 `10 > 5` ? "yes" : "no" → "yes"; Q4 `switch("5")` with `case 5` → "no match" (strict equality); Q5 `if (quantity)` is risky because 0 is falsy, so a valid 0 looks "missing."

◆ OBJECTIVES

- ◆ Write `for` loops that repeat a known number of times
- ◆ Use `while` and `do...while` for condition-based repetition
- ◆ Iterate arrays with `for...of` and object keys with `for...in`
- ◆ Control loops with `break` / `continue`; avoid infinite loops and off-by-one errors

◆ TEACHING SEQUENCE

- 1 **Frame (3m).** "Fifty products, fifty nearly identical lines? No." Introduce DRY — **Don't Repeat Yourself**. A loop writes the instructions once and repeats them.
- 2 **Anatomy of `for` (8m).** Label the three parts on screen and say when each runs:

```
// start;      keep going?; after each pass
for (let i = 0; i < 5; i++) {
  console.log(`Iteration ${i}`);
}
```

Then loop an array by index with `i < fruits.length`, count backward, and count by 2s.

- 3 **while / do...while (5m).** The password-attempts `while`; then the dice-roll `do...while`. "`do...while` always runs its body at least once."
- 4 **for...of and for...in (6m).** `for (const color of colors)` gives values. `for (const key in user)` gives keys. Show `typeof key` is `"string"` on an array to explain why `for...in` is for objects only.
- 5 **break & continue (5m).** "Find the first number over 50" with `break`; "print only evens" with `continue`. `break` leaves the loop; `continue` skips to the next pass.
- 6 **Pitfalls (4m).** Infinite loop (forgot `count++`) and off-by-one (`<=` length). Keep the infinite loop *commented out* — show, don't run.
- 7 **Lab (13m).** Star Rating & Search Filter.
- 8 **Check (3m).** Quiz Q1-Q5.
- 9 **Journal (3m).** When would they pick `for`, `while`, `for...of`, `for...in`, and which loop fits a list from a page they'd like to build.

◆ LIVE-DEMO SCRIPT

```
const items = ["a", "b", "c"]; // indices 0, 1, 2

for (let i = 0; i <= items.length; i++) {
  console.log(i, items[i]);
}

// Ask: "How many lines will print?"
```

Most rooms say three. Run it: four lines, the last is `3 undefined`. Say: "length is 3, but the last index is 2. `<` length, never `<=`." Fix it live. This is quiz Q5.

◆ LAB — STAR RATING & SEARCH FILTER

Part 1: finish `starRating(rating)` so it builds `"*****"` with a `for` loop from 1 to 5. Part 2: loop the five products with `for...of`, show the ones containing `"wireless"` (case-insensitive), and finish with `"3 results found."` Files in `labs/module-2/l06-loops/`.

Set expectations: `starRating` is a function, and functions are next lesson. The starter hands them the shell; they write only the loop. Circulate for: `.toLowerCase()` on *both* sides; the counter declared with `let` outside the loop.

◆ WATCH-OUTS

- ◆ **Off-by-one with `<=`.** `i <= items.length` runs one extra time and reads `undefined`. For stars, the loop is `i = 1; i <= 5` — counting 1-to-5 is fine; indexing an array with `<=` length is not.

- ◆ **Infinite loops freeze the tab.** Teach recovery before anyone needs it: close the tab (not the browser), or pause in DevTools Sources.
- ◆ **Case-sensitive search.** `"Wireless Mouse".includes("wireless")` is `false`. Lowercase both the product and the term.
- ◆ `for...in` on arrays gives string keys (`"0"`, `"1"`), not values. Arrays get `for...of`.
- ◆ **Counter reset inside the loop** — `let matchCount = 0` placed inside the loop body always ends at 0 or 1.

◆ CHECK FOR UNDERSTANDING

Key answers: Q1 `i = 0; i < 3` runs 3 times; Q2 array values → `for...of`; Q3 `continue` skips to the next iteration; Q4 `do...while` always runs its body at least once; Q5 `<= items.length` runs one extra time and logs undefined.

◆ OBJECTIVES

- ◆ Write function declarations and explain that they are hoisted
- ◆ Create function expressions and arrow functions (including implicit return)
- ◆ Use parameters, arguments, and default values; return values and use them in expressions
- ◆ Write small functions that do one thing, with clear names and early returns

◆ TEACHING SEQUENCE

- 1 **Frame (3m).** The recipe analogy: define the recipe card once, cook it (call it) whenever you like, and swap ingredients (parameters) each time.
- 2 **Declarations + calling (6m).** Write `greet()`. Show `console.log(greet)` (the function itself, nothing runs) vs `greet()` (the parentheses run it). Call `sayHi()` above its definition: it works, because declarations are hoisted.
- 3 **Parameters, arguments, defaults (7m).** "Parameters are the parking spots; arguments are the cars." `greet(name = "stranger")`, then `showInfo(1, 2)` to show a missing argument becomes `undefined`.
- 4 **return (7m).** `add(3, 5)` into a variable, then `add(10, 20) * 2`. `return` exits immediately (`checkAge(-5)`). A function with no `return` gives `undefined`.
- 5 **Function expressions (5m).** `const multiply = function(a, b) {...};` Try calling one above its line: `ReferenceError`. Compare with the declaration table.
- 6 **Arrow functions (7m).** Shrink one function three times:

```
const add      = function(a, b) { return a + b; };
const addArrow = (a, b) => { return a + b; };
const addShort = (a, b) => a + b;    // implicit return
```

Then the shortcuts: one parameter, no parameters, and `=> ({ ... })` for objects.

- 7 **Best practices (3m).** Do one thing; descriptive names; short functions; early returns (`getDiscount` before/after).
- 8 **Lab (11m).** Mini Utility Library.
- 9 **Check (3m).** Quiz Q1–Q5.
- 10 **Journal (3m).** One job they repeated in L05–L06, described as a function: name, parameters, return value, declaration or arrow.

◆ LIVE-DEMO SCRIPT

```
const makeUser = n => { name: n };
console.log(makeUser("Ray"));    // Ask: "What prints?"
```

It prints `undefined`. Say: "JavaScript read those braces as a function body with a label called `name`, not as an object — and a body with no `return` gives `undefined`." Wrap it: `n => ({ name: n })` and rerun. That's quiz Q5, and it's the single most confusing arrow rule.

◆ LAB - MINI UTILITY LIBRARY

Four helpers: `capitalize(str)` as a **declaration**, `clamp(value, min, max)` as an **expression**, `pluralize(count, singular, plural = singular + "s")` as an **arrow** with a default parameter, and `temperatureConverter(value, fromUnit)` in any style, rounded with `.toFixed(1)` and returning `"Invalid unit"` otherwise. Files in `labs/module-2/l07-functions/`.

Circulate for: functions that `console.log` instead of `return` (the test line then shows `undefined`); expressions called above their definition; `"0.0"` — `.toFixed(1)` returns a *string*, which is exactly what the template needs.

◆ WATCH-OUTS

- ◆ **Logging instead of returning.** A function that only logs gives `undefined` to its caller. `console.log(greet())` prints the greeting *and then* `undefined`.

- ◆ **Hoisting: declarations vs `const` expressions.** A declaration can be called above its line. A `const` / `let` function expression called early throws `ReferenceError: Cannot access '...' before initialization` — not a `TypeError`. (Only the old `var` form gives `TypeError: ... is not a function`.)
- ◆ **Forgetting the parentheses.** `greet` is the function; `greet()` runs it.
- ◆ **Braces on an arrow turn off implicit return.** `(a, b) => { a + b }` returns `undefined`. Either drop the braces or add `return`.
- ◆ **Default parameters only fill `undefined`, not `null` or `""`.** Mention it if someone asks; don't lead with it.

◆ CHECK FOR UNDERSTANDING

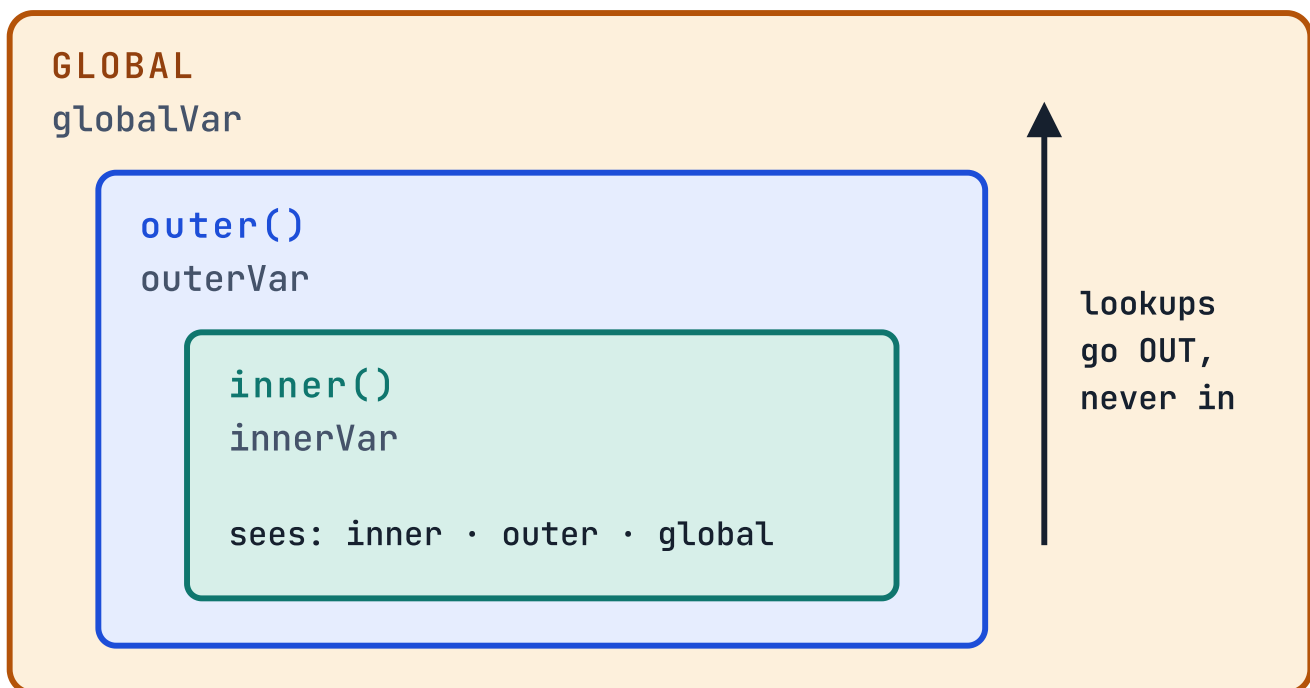
Key answers: Q1 no `return` → `undefined`; Q2 callable before its definition → **function declaration**; Q3 `fn = n => n * 3; fn(4)` → 12; Q4 `greet()` with default `"friend"` → `"Hi, friend!"`; Q5 arrow returning an object → `n => ({ name: n })`.

◆ OBJECTIVES

- ◆ Explain global, function, and block scope; predict where a variable is accessible
- ◆ Trace a lookup outward through the scope chain; recognize shadowing
- ◆ Describe how `var`, `let` / `const`, and functions are hoisted; explain the Temporal Dead Zone
- ◆ Explain a closure and build one that keeps a private variable

◆ TEACHING SEQUENCE

- 1 **Frame (3m).** Scope answers one question: "Can I use this variable *here*?" Most "why is this undefined?" mysteries are scope.
- 2 **Global & function scope (4m).** `appName` readable inside a function; `taxRate` inside `calculateTotal` is a `ReferenceError` outside. Two functions, two independent `greeting` variables. "Privacy is a feature."
- 3 **Block scope and the `var` leak (6m).** `let` stays in its `{ }`; `var` leaked escapes the `if`. Show `for (var j...)` leaving `j = 3` behind. Mention the classic `setTimeout 3, 3, 3` bug as a story, not a deep dive.
- 4 **Scope chain & shadowing (6m).** Draw the nested boxes (diagram below): inner looks outward, never inward. Then `color` "red" inside `paint()` vs "blue" outside.
- 5 **Hoisting & the TDZ (7m).** `var nickname` logs `undefined` before its line; `let age` throws a `ReferenceError`; declarations are fully hoisted; `const` function expressions are in the TDZ.
- 6 **Closures (5m).** `createGreeter("Hello")`, then `createCounter()`: `count` survives after the outer function finishes, and nobody can reach it except the returned methods.
- 7 **Lab (8m).** Scope Detective & Score Tracker.
- 8 **Check (3m).** Quiz Q1–Q5.
- 9 **Journal (3m).** Explain the `var` leak and the TDZ without looking back; explain the counter closure to a friend.



Draw this on the board: each scope can see outward to its parents, never inward to its children.

◆ LIVE-DEMO SCRIPT

```
console.log(nickname); // ?
var nickname = "Ray";

console.log(age);      // ?
let age = 25;
```

Take predictions for each line, then run. `undefined`, then `ReferenceError: Cannot access 'age' before initialization`. Say: "Both were hoisted. `var` got a placeholder `undefined`; `let` is in the Temporal Dead Zone until its line runs — and that error is a favor, because a silent `undefined` is harder to debug." That's quiz Q1 and Q4 together.

◆ LAB — SCOPE DETECTIVE & SCORE TRACKER

Part 1: students write a prediction for all eight `console.log` lines *before* the first reload, then compare. Part 2: `createScoreTracker()` returns `addPoints`, `penalty` (never below 0), `getScore`, and `reset` over a private `let score`. Files in `labs/module-2/108-scope-closures/`. Run the debrief as a class: go line by line and have students hold up fingers for how many they got right. Lines 3 and 5 are the ones to stop on. Circulate for: `penalty` using `Math.max(0, score - points)`; `let score` declared *inside* the function, not globally.

◆ WATCH-OUTS

- ◆ Don't demo with `var name`. In a browser script, a global `var name` collides with the built-in `window.name`, which turns every value into a string and confuses the hoisting demo. That's why the lesson uses `nickname`. Stick to it when you improvise.
- ◆ TDZ error is a `ReferenceError`, not a `TypeError`. Calling a `const` function expression before its line: "Cannot access 'greet' before initialization." Only `var greet = function...` produces "greet is not a function."
- ◆ "`var d` is inside the `if`, so line 5 must fail." The most common wrong prediction: `var` ignores blocks and belongs to the whole function.
- ◆ Thinking shadowing changes the outer variable. The inner `const color = "red"` is a different variable; the global stays `"blue"`.
- ◆ Closures sound abstract. Keep it concrete: "the function packs a backpack with the variables it could see when it was made." Mastery isn't the goal today; recognizing the pattern is.

◆ CHECK FOR UNDERSTANDING

Key answers: Q1 `console.log(x); var x = 10;` → `undefined`; Q2 block-scoped keyword → `let`; Q3 a closure is a function that remembers variables from its outer scope; Q4 `console.log(total); let total = 50;` → `ReferenceError: Cannot access 'total' before initialization`; Q5 shadowed `color` → `"blue"`.

CLOSING THE MODULE

MODULE 2 WRAP-UP (10 MIN)

Put the Grade Calculator on the projector and ask the room how they'd rewrite it now that they know functions: `getLetterGrade(score)`, `getMessage(letter)`, an early- `return` guard. That one refactor ties the whole module together. Recap the arc: **decide** → **repeat** → **package** → **scope**. Preview Module 3: "Your programs can decide and repeat. Next we give them real data to work with — arrays, objects, and strings."

Module 3 — Working with Data

How to teach the four lessons that move students from single values to real data: arrays, objects, the map/filter/reduce toolkit, and strings. Timing, live-demo scripts, the misconceptions to pre-empt, and how to run each lab.

4 lessons

~3.3 hrs teaching

Lessons L09-L12

Level Beginner → Intermediate

Maps to Day 2 PM (L09) → Day 3 AM

HOW TO USE THIS GUIDE

- Each lesson has the same rhythm: **Frame** (why it matters) → **Teach** (live-code the concept) → **Lab** (students build) → **Check** (quiz + recap). The minute timings cover teaching, the quiz, and the lesson's **Learning Journal** prompt (in the lesson and the workbook); lab floor time sits on top of that. Budget 25–35 minutes per lab in this module and adjust ±10 min to your group.
- Live-code everything, and predict before you run.** Type the code in front of the class, and before each `console.log` ask the room "what will this print?" This module is full of surprises (`[40, 100, 1].sort()` , `{x:1} === {x:1}` , `32.400000000000006`). A wrong prediction the class sees for itself sticks better than a warning.
- Blue blocks are concepts; teal blocks are hands-on; amber blocks are watch-outs — the errors that eat class time if you don't pre-empt them.
- Quiz answers here are for you. The student workbook has a 3-question subset of each lesson quiz, without answers, labelled with the lesson's question numbers; the separate **Answer Key** covers every question with matching numbers for grading.

BEFORE YOU START THE MODULE

Students need VS Code with Live Server, a browser with DevTools, and the Module 2 habits: writing functions, arrow functions, and `for / for...of` loops. Callbacks carry this whole module, so if arrow functions are shaky, spend five minutes rewriting `function (n) { return n * 2; }` as `n => n * 2` before L09. Copy `labs/module-3/` to the student machines (starters only). If anyone is behind on setup, pair them up; don't stop the room to fix one machine.

MODULE AT A GLANCE

LESSON	TOPIC	TIME	STUDENTS LEAVE ABLE TO...
L09	Arrays	50m	Store, change, search, and loop over an ordered list
L10	Objects	50m	Model a real thing with named properties and methods
L11	Array Methods	55m	Chain filter → sort → map (and reduce) instead of loops
L12	Strings & Template Literals	40m	Search, slice, clean, and format text

The labs are **four stand-alone data exercises**, not one cumulative project: a to-do list, a contact book, a product-catalog pipeline, and a text-formatting toolkit. The lesson exercises are console exercises, so each lab page shows its output on the page as well as in the Console (via a provided `show()` helper). Students still get the full DevTools habit, and nobody gets stuck because they can't find the Console. Day 2 ends after L09; L10–L12 fill the Day 3 morning.

L09 Arrays

OBJECTIVES

- Create arrays with literal syntax (and know why to avoid `new Array(3)`)
- Read and update elements by index; use `length` and `at(-1)`
- Add and remove with `push` / `pop` / `unshift` / `shift`; insert, remove, or replace with `splice`
- Search with `includes`, `indexOf`, `find`; copy and combine with `slice`, `concat`, and spread
- Loop with `for`, `for...of`, and `forEach`

TEACHING SEQUENCE

- 1 **Frame (4m).** Write `const song1 = ...; const song2 = ...; const song3 = ...;` and ask "what about a playlist of 500?" An array is one variable holding an ordered list. Draw numbered slots on the board, starting at 0.
- 2 **Create, read, update (8m).** Literal syntax, index access, `animals[99] → undefined` (no error!), last element via `length - 1` and `at(-1)`, and a quick array destructure `const [r, g, b] = rgb`. Show that `const` arrays can still be `push`ed: `const` locks the variable, not the contents.
- 3 **The four end methods (6m).** `push`/`pop` at the end, `unshift`/`shift` at the start. Draw the array as a line of people: `push` = join the back, `shift` = serve the front. Point out that `push` returns the new *length* and `pop` / `shift` return the removed *element*.
- 4 **`splice` vs `slice` (10m).** The lesson's hardest ten minutes. Run both on the same array and log the original after each:

```
const letters = ["a", "b", "c", "d", "e"];
letters.slice(1, 3); // ["b", "c"] - letters unchanged
letters.splice(1, 2); // ["b", "c"] - letters is now ["a", "d", "e"]
```

Same return value, completely different side effect. Mnemonic: *splice has a "p" for "permanent"*. Then show insert `splice(1, 0, "x")` and replace `splice(1, 1, "x")`.

- 5 **Searching (6m).** `includes` → true/false, `indexOf` → position or `-1`, `find` / `findIndex` with a condition. First look at a callback here: `numbers.find(num => num > 10)`. Read it aloud: "find the first num where num is greater than 10."
- 6 **Combine & copy (5m).** `concat`, then spread `[...front, ...back]` as the modern way; `[...original]` as a copy. Show the lesson's mutating vs non-mutating list.
- 7 **Iterating (5m).** Same array three ways: `for` (need the index), `for...of` (default), `forEach` (callback with element + index; no `break`). End on the tax loop printing `32.400000000000006`. Name it and park it: "floating point. We'll format money with `toFixed(2)`."
- 8 **Lab launch (2m).** Hand off to the To-Do List Manager.
- 9 **Check & journal (4m).** Quiz poll, then the journal prompt (`splice` vs `slice` in their own words).

LIVE-DEMO SCRIPT

- 1 Open the Console and type `const todos = ["Buy groceries", "Walk the dog"]`. Ask: "How do I get the dog walk?" Take answers. Someone will say `todos[2]`. Run it: `undefined`. That's the zero-index lesson, learned the memorable way.
- 2 `todos.push("Read")`. Point out the Console echoes `3`, the new length, not the array.
- 3 `todos.indexOf("Walk the dog") → 1`, then `todos.splice(1, 1)`. This is exactly the `removeTodo` recipe from the lab. Say so.
- 4 `todos.indexOf("Nap") → -1`. "Not found is -1, and that's why we check for it before splicing." Then show why: `todos.splice(-1, 1)` quietly removes the *last* item.

LAB - TO-DO LIST MANAGER

Students write `addTodo`, `removeTodo`, `insertTodoAt`, `listTodos`, and `completeTodo` on a shared `todos` array, then uncomment the tests. Starter in `labs/module-3/l09-arrays/starter/`; solution in `labs/module-3/l09-arrays/solution/`. Tell students to use `show()`

where the lesson says `console.log()` : it logs *and* prints on the page.

Circulate for: the `-1` check before `splice` in `removeTodo`; `splice(index, 0, task)` (zero delete count) for inserting; `i + 1` in the numbered list; `shift` (not `pop`) in `completeTodo`. Early finishers: add a `moveTodo(list, from, to)` using two splices.

◆ WATCH-OUTS

- ◆ **splice vs slice.** The #1 confusion of the lesson (quiz Q3 targets it). If a student's array "mysteriously lost items," look for a `splice` where they meant `slice`.
- ◆ **Off-by-one.** The last index is `length - 1`; `arr[arr.length]` is always `undefined`. In loops, `i <= arr.length` runs one step too far.
- ◆ `if (list.indexOf(x))`. Wrong in both directions: "found at index 0" is falsy and "not found" (`-1`) is truthy. Compare to `-1` explicitly, or use `includes` for yes/no questions.
- ◆ "But it's `const` !" Students expect `const` arrays to be frozen. `push` works; only `todos = [...]` fails.
- ◆ **Expecting `push` to return the array.** `const list = todos.push("x")` gives a number. That's why the lab functions `push` and then `return list` on its own line.

◆ CHECK FOR UNDERSTANDING

Run quiz Q1–Q4 as a quick poll. Key answers: first index = A) 0; add to the end = B) `push()`; `splice()` = C) removes, inserts, or replaces at any position; `["a", "b", "c"].includes("b")` = B) `true` (a boolean, not the index).

◆ JOURNAL (3–5 MIN)

Prompt: a list they'd like to build, which array methods they'd use on it, and splice vs slice in their own words. Read a few splice/slice answers aloud. They're a fast diagnostic for who needs a two-minute recap tomorrow morning.

◆ OBJECTIVES

- ◆ Create objects with literal syntax (including shorthand properties)
- ◆ Read and write properties with dot and bracket notation; add, update, delete
- ◆ Write methods and use `this`, and explain why arrow functions don't work as methods that need it
- ◆ Nest objects and arrays; work with arrays of objects
- ◆ Destructure objects; iterate with `Object.keys/values/entries`
- ◆ Check for properties (`in`, `Object.hasOwnProperty`, `?.`) and explain reference equality

◆ TEACHING SEQUENCE

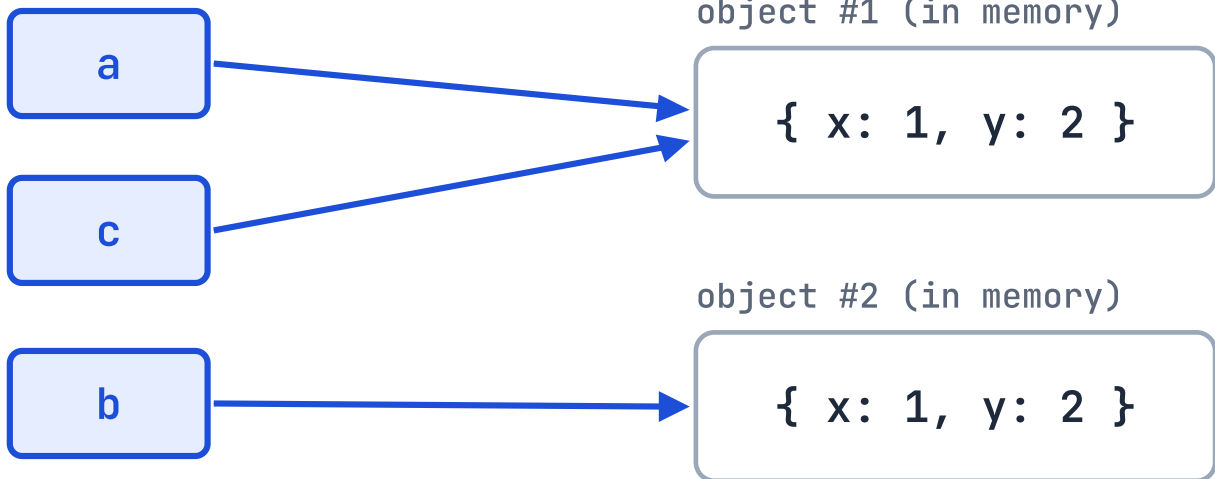
- 1 **Frame (4m).** Recap day 2: "Arrays store by *position*. What's at `user[2]`? No idea." Objects store by *name*: `user.email` says what it is.
- 2 **Create & access (8m).** Object literal, key rules (quotes for `"price-usd"`), shorthand `{ name, age }`. Dot notation by default; bracket notation for odd keys and *keys in variables*:

```
const field = "name";
product[field];    // "Headphones" – uses the VALUE of field
product.field;     // undefined – looks for a key literally called "field"
```

- 3 **Add, update, delete (4m).** Assign to create, `delete` to remove, and the same `const` lesson as arrays.
- 4 **Methods & `this` (9m).** Build the `player` object live and call `addPoints` twice. `this` = "the object before the dot." Then run the lesson's `broken.greet()` arrow version and show the empty name. Arrow functions don't get their own `this`. Rule for this course: **methods use the shorthand `greet() { }` syntax; callbacks use arrows.**
- 5 **Nesting & arrays of objects (6m).** `student.address.city`, `student.grades["CS 101"]`. Then the spreadsheet analogy: an array of objects is a table. Each object is a row, each key a column. This is the shape of the next two labs, and of most real-world data they'll handle.
- 6 **Destructuring (5m).** Basic, rename (`user_name: username`), defaults, and in function parameters, which is the form they'll see most.
- 7 **Iterating & spread (5m).** `Object.keys`, `values`, `entries` with `for (const [key, value] of ...)`. Merge settings with `{ ...defaults, ...userPrefs }`: later wins. Mention "shallow copy" once.
- 8 **Checking & comparing (4m).** `"email" in user` vs `!== undefined`; optional chaining `user.work?.zip`. Then the predict-first moment: `{ x: 1 } === { x: 1 }`. Draw the reference diagram below.
- 9 **Lab launch (2m).** Hand off to the Contact Book.
- 10 **Check & journal (3m).**

◆ BOARD DIAGRAM – COMPARED BY REFERENCE

variables



`a === c → true`

`a === b → false`

=== on objects asks "same object?", not "same contents?"

◆ LIVE-DEMO SCRIPT

- 1 In the Console: `const a = { x: 1 }; const b = { x: 1 }; const c = a;` Ask for predictions on `a === b` and `a === c`. Run both.
- 2 `c.x = 99;` then log `a.x`. Gasps are normal. "c isn't a copy, it's a second name for the same object." Then `const d = { ...a }; d.x = 5;` and `a.x` is still 99. That's a real copy.
- 3 Tie it to the lab: `findContact` returns the *actual* object in the array, which is why `Object.assign(contact, updates)` in `updateContact` changes the contact book itself.

◆ LAB — CONTACT BOOK

Students write `addContact`, `findContact`, `updateContact`, `displayContact`, and `listByGroup` over an array of contact objects. Files in `labs/module-3/l10-objects/`.

Set expectations: the lesson's solution uses `.map()`, `.filter()`, and `.join()`, which come in L11–L12. Students may solve `listByGroup` with `for...of` + `push` and `displayContact` with a template literal. Accept both. The hint says so. Circulate for: `return contact || null` (not `undefined`); returning `true` / `false` from `addContact`. Note: the lesson's TODO test calls `listByGroup(contacts, "work")` while its solution calls `"family"`. The lab solution shows both (`["Bob", "Dave"]` and `["Carol"]`, since Alice moved to `friend`).

◆ WATCH-OUTS

- ◆ **Arrow functions as methods.** `greet: () => this.name` gives `undefined` or an empty string, not an error, so it's hard to spot. Quiz Q3 targets it.
- ◆ **Reference equality.** `{x:1} === {x:1}` is `false`; the same goes for arrays (`[1] === [1]`). Students later write `if (contact === { name: "Bob" })` and wonder why it never matches. Compare a property (`c.name === "Bob"`) instead.
- ◆ **`obj.key` when the key is in a variable.** Always `undefined`. If a lookup "returns nothing," check for dot notation on a variable.
- ◆ **Missing commas between properties** in object literals, and trailing semicolons inside them. The `SyntaxError` line number points to the *next* property.
- ◆ **"Shallow" copies.** Spread copies one level; nested objects are still shared. Mention it; don't drill it.

◆ CHECK FOR UNDERSTANDING

Key answers: key in a variable = **B**) **bracket notation**; `Object.keys({a:1,b:2,c:3}) = B`) `["a", "b", "c"]`; arrow methods = **B**) **arrow functions don't have their own this**; `{ x: 1 } === { x: 1 } = B`) `false` (compared by reference).

◆ JOURNAL (3–5 MIN)

Prompt: describe something as an object (four properties + one method), when to use bracket notation, and what still feels fuzzy about `this` or destructuring. If several people name `this`, that's expected. It keeps coming back, and it gets easier each time.

◆ OBJECTIVES

- ◆ Transform with `map`, keep matches with `filter`, collapse with `reduce`
- ◆ Answer questions with `find`, `some`, and `every`
- ◆ Sort numbers and objects with a compare function; know that `sort` mutates
- ◆ Chain methods into a filter → sort → map pipeline and pick the right method for a task

◆ TEACHING SEQUENCE

- 1 **Frame (4m)**. Put the L09 tax loop on screen (empty array, `for...of`, `push`), then replace it with one line: `prices.map(price => price * 1.08)`. "You describe *what* to do to each item; the method does the looping."
- 2 **forEach returns undefined (3m)**. `const shouted = names.forEach(...) → undefined`. `forEach` is for side effects (logging, later the DOM), not for building arrays.
- 3 **map (7m)**. Same length out as in. Strings from objects (`users.map(u => u.name)`), then the wrapped-object return `u => ({ ...u, birthYear: ... })`. Point at the parentheses: without them, the braces are a function body.
- 4 **filter (6m)**. Callback returns true = keep. Passing scores, in-stock products, and "remove all Walk dog" as the non-mutating alternative to splice.
- 5 **reduce (10m)**. Slow down. Run the lesson's logging version so the Console prints every step:

```
numbers.reduce((sum, num) => {
  console.log(`sum: ${sum}, num: ${num}`);
  return sum + num;
}, 0); // ← the initial value (quiz Q3)
```

Act it out: the accumulator is a running total on a sticky note that gets handed from item to item. Then show the tally pattern (`{}` as the initial value) so they see `reduce` can build any value, not just a sum.

- 6 **find, some, every (5m)**. Questions, not transformations. `some` = "any?", `every` = "all?". Both stop early.
- 7 **sort (7m)**. Predict first: `[40, 100, 1, 5, 25].sort()`. Run it: `[1, 100, 25, 40, 5]`. The default sort compares *as strings*. Fix with `(a, b) => a - b`; flip for descending; `localeCompare` for names. Then show it mutated the original, and offer `toSorted()` / `[...arr].sort()`.
- 8 **Chaining (6m)**. Build the electronics pipeline one method at a time, logging after each step. Read it top-down as a sentence. Filter first, because after `map` to names the price is gone.
- 9 **Lab launch (2m)**. "No loops allowed" is the rule.
- 10 **Check & journal (5m)**.

◆ LIVE-DEMO SCRIPT

- 1 Paste the lab's 8-product array into the Console (it's in the starter). Ask for "just the names." Type `products.map(p => p.name)`.
- 2 "Only in-stock." Add `.filter(p => p.inStock)` before the map and run again. Count: 6.
- 3 "Total value of in-stock." `products.filter(p => p.inStock).reduce((sum, p) => sum + p.price, 0) → 374.94000000000005`. Ask "is our math wrong?" No: decimals like 29.99 can't be stored exactly in binary. Add `.toFixed(2) → "374.94"`, and point out the quotes: `toFixed` returns a *string*, meant for display. Do math on the raw numbers and format at the end.
- 4 Deliberately break it: `.map(p => { p.name }) → an array of undefined`. Curly braces need `return`.

◆ LAB – PRODUCT CATALOG PIPELINE

Six challenges on an 8-product array, each solved with array methods only. Files in `labs/module-3/l11-array-methods/`. Each challenge has its `show()` line pre-written and commented out. Students write the `const`, then uncomment.

Expected output to check at a glance: 8 names · 6 in stock · [Mousepad XL, Wireless Mouse, Mechanical Keyboard] · total "374.94" · every → true · a 6-line sale list from "Mousepad XL — \$11.99 (was \$14.99)" to "Headphones — \$119.99 (was \$149.99)". Challenge 6 is the stretch (filter → map → sort → map). Let strugglers stop at 5 and walk through 6 together at the end.

◆ WATCH-OUTS

- ◆ **Default `sort()` on numbers** sorts alphabetically: `[1, 100, 25, 40, 5]`. Numbers always need `(a, b) => a - b`.
- ◆ **`sort` mutates**. Quiz Q4. In the lab it's safe because `filter` already made a new array. Sorting `products` directly reorders the catalog for every later challenge. The solution page ends by proving the original order survived.
- ◆ **Floating-point output**. `374.94000000000005` and `32.400000000000006` aren't bugs. Format with `toFixed(2)` for display, and remember it returns a string (`"374.94" + 1` is `"374.941"`).
- ◆ **Braces without `return`** in a callback, giving `[undefined, undefined, ...]`. And returning an object literal needs `({ ... })`.
- ◆ **`reduce` without an initial value**. It works for sums on non-empty arrays, then crashes on `[]`, and misbehaves on arrays of objects (the first accumulator is a whole product). Always pass the second argument.

◆ CHECK FOR UNDERSTANDING

Key answers: `map()` returns **B) a new array with the same number of elements**; `[1,2,3].filter(n => n > 1)` = C) `[2, 3]`; `reduce`'s second argument = **B) the initial value of the accumulator**; modifies the original = C) `sort()`.

◆ JOURNAL (3-5 MIN)

Prompt: a plain-English filter → sort → map pipeline on their own data, why filter comes first, and what's still unclear (the lesson itself guesses reduce).
Normalize it: "Most developers needed a few tries before reduce clicked."

◆ OBJECTIVES

- ◆ Explain that strings are immutable: every method returns a new string
- ◆ Use template literals for interpolation and multiline text
- ◆ Search (`includes` , `startsWith` , `endsWith` , `indexOf`) and extract (`slice` , `at`)
- ◆ Transform (`toUpperCase` , `trim` , `replace` / `replaceAll`) and split/join
- ◆ Format with `padStart` , `padEnd` , and `repeat`

◆ TEACHING SEQUENCE

- 1 **Frame: immutability (3m).** `const g = "hello"; g.toUpperCase(); g` is still lowercase. "String methods hand you a new string. Catch it in a variable." Contrast with `push` / `splice` , which changed arrays.
- 2 **Template literals (4m).** Quick review of `${}` with an expression and a ternary, then multiline. Note the lesson's warning: user text in an HTML template must be escaped before it goes near `innerHTML` (the next module comes back to this).
- 3 **Searching (4m).** `includes` is case-sensitive, so use `toLowerCase()` first. `endsWith(".pdf")` for file types.
- 4 **Extracting (5m).** String `slice` works exactly like array slice, including negatives. `"Hello, World!".slice(-6)` : predict, then run. Mention `substring` exists and prefer `slice` . `at(-1)` for the last character.
- 5 **Transforming (6m).** Case, `trim` ("always trim user input"), `replace` (first only) vs `replaceAll` . Show `phone.replace(/\D/g, "")` as a preview only. See the regex primer below.
- 6 **split / join (6m).** Opposites. Then the split → map → join pattern with `titleCase` . They already know `map` from this morning.
- 7 **Padding & repeat (4m).** Order numbers `"42".padStart(6, "0")` , the card mask, the progress bar.
- 8 **Lab launch + regex primer (4m).** See below. Do it *before* students open the starter.
- 9 **Check & journal (4m).** Five quiz questions in this lesson. Then close Module 3.

◆ REGEX PRIMER: SAY THIS BEFORE THE LAB

The lesson only *previews* regular expressions, but the lab's `slugify` , `countWords` , and `initials` use them. Without a minute of explanation, students copy them blindly or freeze. Put these on the board and read each one aloud:

```
/\s+/g      // one or more (+) whitespace characters (\s), everywhere (g)
/~/g        // one or more hyphens, everywhere
/[^a-z0-9\s-]/g // any ONE character NOT (^) in the set [a-z, 0-9, space, hyphen]
/\D/g       // any non-digit (from the lesson's phone example)
```

Frame: "A regex is a pattern between slashes. You'll read these today, not write your own. `split(/\s+/)` splits on any run of spaces, which is why "lots of spaces" gives 3 words instead of empty strings." Show the difference live: `"a b".split(" ")` → `["a", "", "b"]` vs `"a b".split(/\s+/)` → `["a", "b"]` . Curious students can explore patterns at regex101.com.

◆ LAB - TEXT FORMATTING UTILITY

Students write `truncate` , `slugify` , `countWords` , `initials` , and `maskEmail` . Files in `labs/module-3/l12-strings/` . The starter's `slugify` TODO includes a plain-English reading of the two regexes it needs. The solution page prints each call next to its result, with quotes so stray spaces show.

Suggested order: `truncate` → `maskEmail` → `initials` → `countWords` → `slugify` (easiest to hardest). Circulate for: `slice(0, maxLength - 3)` (the "..." counts toward the length: `"Hello..."` is 8); `countWords("")` returning 0, not 1; destructuring `const [local, domain] = email.split("@")` (L09/L10 skills coming back).

◆ WATCH-OUTS

- ◆ **Calling a method and throwing the result away.** `name.trim();` on its own line does nothing. Strings are immutable (quiz Q1).
- ◆ **Regex before it's taught.** Covered by the primer above. Don't let it turn into a regex lesson; five minutes maximum.

- ◆ **Quotes vs backticks.** `"Hello ${name}"` prints the dollar sign literally. Only backticks interpolate.
- ◆ `replace` only changes the first match. Use `replaceAll` (or a `g` regex).
- ◆ **Negative slice arguments.** `slice(-6)` counts from the end (quiz Q4). `substring` treats negatives as 0, another reason to stick with `slice`.

◆ CHECK FOR UNDERSTANDING

Key answers: strings are **B) immutable: methods always return new strings**; `"hello world".split(" ") = B) ["hello", "world"]`; ends with ".pdf" = **B) `str.endsWith(".pdf")`**; `"Hello, World!".slice(-6) = D) "World!"`; `"7".padStart(3, "0") = A) "007"`.

◆ JOURNAL (3-5 MIN)

Prompt: a string function for text on their own site, a prediction for `"JavaScript".slice(-6)` (it's `"Script"`), and the method they'll use most. Have them check the prediction in the Console before moving on.

CLOSING THE MODULE

MODULE 3 WRAP-UP (10 MIN)

Put the L11 solution page on the projector and ask the room to name the method behind each line of output. Recap the arc: **lists (arrays) → named things (objects) → transforming lists (map/filter/reduce) → text (strings)**. Then the bridge: "Everything so far has lived in the Console and in a helper we told you to ignore. That helper puts text on the page with `document.querySelector` and `textContent`". In Module 4 you'll learn exactly how, and your data will drive real pages."

Module 4 — The DOM

How to teach the four lessons where JavaScript finally touches the page: the document tree, selecting and changing elements, responding to events, and building elements from scratch. Timing, live-demo scripts, the misconceptions to pre-empt, and how to run each lab.

4 lessons

~3.3 hrs teaching

Lessons L13-L16

Level Intermediate

Maps to Day 3 PM (L13-L14) → Day 4 AM

HOW TO USE THIS GUIDE

- Each lesson has the same rhythm: **Frame** (why it matters) → **Teach** (live-code the concept) → **Lab** (students build) → **Check** (quiz + recap) → **Journal**. The minute timings in each teaching sequence add up to the lesson's length; adjust ±10 min to your group. Close each lesson with 3–4 minutes for the lesson's **Learning Journal** prompt (in the lesson and the workbook).
- Live-code everything, with the console open.** Split the projector: editor on the left, browser + DevTools on the right. Every DOM change should be something students see happen, and every error something they see you read and fix.
- Blue blocks are concepts; teal blocks are hands-on; amber blocks are watch-outs — the errors that eat class time if you don't pre-empt them.
- Quiz answers here are for you. The student workbook has a 3-question subset of each lesson quiz, without answers, labelled with the lesson's question numbers; the separate **Answer Key** covers every question with matching numbers for grading.

BEFORE YOU START THE MODULE

Students need VS Code with Live Server, a browser with DevTools, and comfort with functions, arrays, objects and template literals from Modules 1–3. Open `labs/module-4/` and check each `starter/` folder opens with Live Server on your machine. From this module on, **every lab is a real page**: an `index.html` that loads `script.js` at the end of `<body>`. If anyone's script "does nothing," check the `<script src>` path and the Console before anything else.

MODULE AT A GLANCE

Lesson	Topic	Time	Students leave able to...
L13	Introduction to the DOM	45m	Explain the document tree and walk it from the console
L14	Selecting & Modifying Elements	50m	Select elements and change text, classes, styles, attributes
L15	Events & Event Listeners	55m	Respond to clicks, keys and submits; delegate from a parent
L16	Creating & Removing Elements	50m	Build, insert, clone and remove elements safely

Each lab is a **standalone page** (no cumulative project this module): explore a page → edit a profile card → a task list → a notification system. The skills stack, though — the L16 notification system uses everything from L13–L15. Day 3 ends after L14; open Day 4 with a five-minute recap of selecting and modifying before L15.

L13 Introduction to the DOM

45 min

◆ OBJECTIVES

- ◆ Explain what the DOM is and why JavaScript works on it, not on the HTML file
- ◆ Describe how the browser builds the document tree; identify `window`, `document`, elements and nodes
- ◆ Navigate with `parentElement`, `children`, and the `*ElementSibling` properties
- ◆ Inspect the live DOM in DevTools and tell it apart from View Source

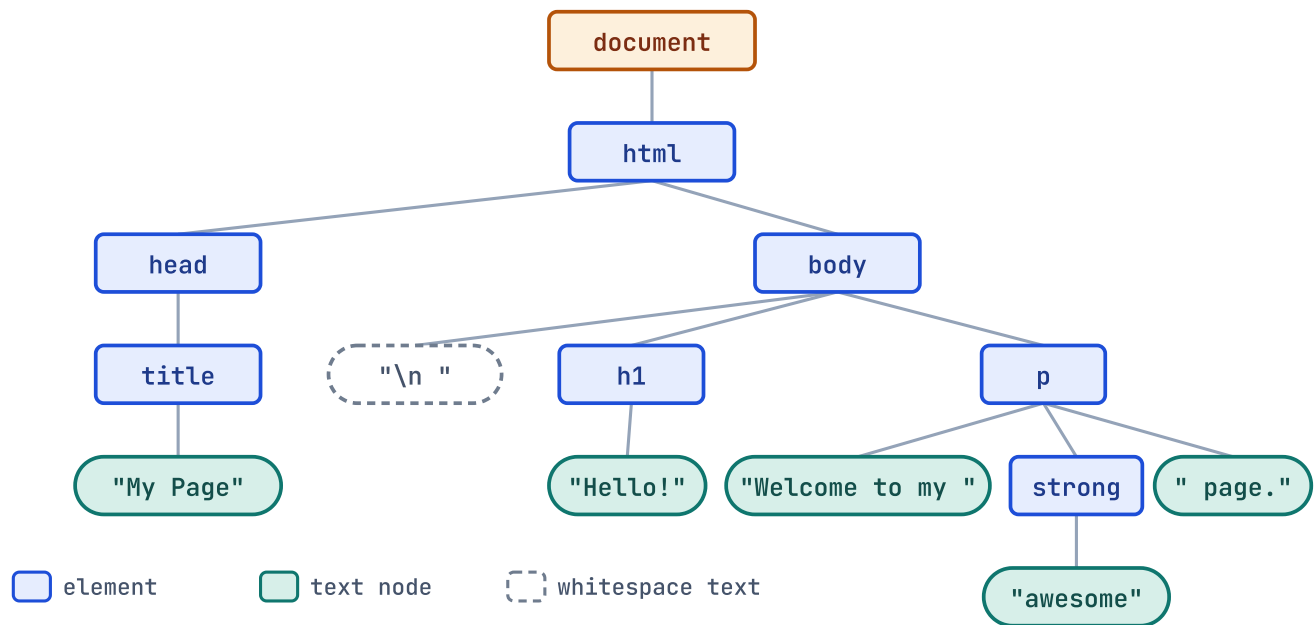
◆ TEACHING SEQUENCE

- 1 **Frame (4m).** "Everything you've written so far only talked to the console. Today JavaScript touches the page." Give the lesson's analogy: *HTML is the blueprint, the DOM is the building, JavaScript is the contractor who remodels it after it's built.*
- 2 **Draw the tree (7m).** Put the lesson's small HTML page on screen and draw its tree on the board (or show the diagram below). Name **parent**, **child**, **sibling**, **text node** on the drawing. Point at the dashed whitespace node — it's the reason for the watch-out later.
- 3 **Key objects & node types (6m).** In the console: `window.innerWidth`, `document.title`, `document.body`. Then `h1.nodeType` (1) vs `h1.firstChild.nodeType` (3). "`document` is your front door to the page."
- 4 **Traversing (8m).** Live-walk a list:

```
const list = document.querySelector("ul");
list.children;           // HTMLCollection of <li>
list.firstChild;         // the first <li>
list.firstChild;         // #text "\n " - surprise!
list.children[1].nextElementSibling;
```

Let the `firstChild` surprise land, then show the Element-vs-Node table: "use the Element versions."

- 5 **DevTools & source vs live DOM (5m).** Change the `h1` in the console, then open View Source (Ctrl + U): unchanged. Open Elements: changed. Show `$0`.
- 6 **Lab (8m).** DOM Explorer — see below.
- 7 **Check (3m).** Quiz Q1–Q5 as a quick poll.
- 8 **Journal (4m).** The lesson's prompt: sketch a DOM tree of one of their own pages.



The DOM tree for the lesson's page. Line breaks between tags become whitespace text nodes (dashed).

◆ LIVE-DEMO SCRIPT

- 1 Open `labs/module-4/l13-dom-explorer/starter/index.html` with Live Server, F12, Console. Type `document.body.children` and expand it: "five element children — count them in the HTML."
- 2 Type `document.querySelector("#family").firstChild`. Read the output aloud: `"#text"`. Ask the room why. Then `.firstElementChild`.
- 3 Set `document.querySelector("h1").textContent = "Changed!"`, then Ctrl + U. "The file didn't change. The *DOM* did." Refresh to undo.

◆ LAB — DOM EXPLORER

In the lesson, students explore the lesson page itself. For class, use the practice page in `labs/module-4/l13-dom-explorer/starter/`: its `README.md` is the worksheet (the lesson's 10 console tasks + 3 "think about it" questions). The solution in `labs/module-4/l13-dom-explorer/solution/` runs every task and prints the results on the page and in the console — project it for the debrief.

Circulate for: typing into the Console (not the Elements search box); forgetting the `.` in `".lesson-section"`; students who can't find `<main>` at `children[3]` — have them count `<body>`'s element children aloud.

◆ WATCH-OUTS

- ◆ **Whitespace text nodes.** `firstChild`, `childNodes` and `nextSibling` include the line breaks between tags. Beginners assume `firstChild` is the first tag. Rule: *if you want an element, use the property with "Element" in its name.*
- ◆ **"I edited the DOM, why didn't my file change?"** Console and Elements-panel edits live in memory only. Refresh throws them away. Real changes go in the `.js` file.
- ◆ **Script in `<head>` without `defer`** → `document.querySelector` returns `null` because the body hasn't been parsed yet. Keep scripts at the end of `<body>` (or use `defer`).
- ◆ **`children` is a live `HTMLCollection`**, not an array — it has `length` and indexes but no `forEach`. Plant the seed; L14 contrasts it with the static `NodeList`.

◆ CHECK FOR UNDERSTANDING

Key answers: the DOM is **B** — an in-memory tree of objects; entry point = **B** — `document`; `parentElement` = **B** — the element that contains this one; a JS-added `<p>` shows in **C** — the Elements panel (not View Source); `list.firstChild` isn't the `<li>` because **D** — the whitespace before it is a text node.

◆ OBJECTIVES

- ◆ Select with `querySelector`, `getElementById` and `querySelectorAll`
- ◆ Change text safely with `textContent`; know when `innerHTML` is dangerous
- ◆ Add, remove and toggle classes with `classList`; set inline styles with `style`
- ◆ Read and write attributes and `data-*` values with `dataset`

◆ TEACHING SEQUENCE

- 1 **Frame (3m).** "Last lesson we looked. Now we grab and change." Hook: "If you know CSS selectors, you already know how to find elements."
- 2 **Selecting one (7m).** `querySelector("h1")`, `( ".card" )`, `( "#main-nav" )`, an attribute selector. Then a typo'd selector → `null`. Then `null.textContent` → the famous *"Cannot read properties of null"*. Read that error together.
- 3 **Selecting many (6m).** `querySelectorAll` returns a `NodeList`: `forEach` works, `map` doesn't. Spread it: `[...items].map(...)`. Show that it's a *snapshot*: add an `<li>`, the old `NodeList`'s `length` doesn't change; `list.children.length` does.
- 4 **Text & the XSS talk (8m).** `textContent = "<em>Not italic</em>"` shows the tags literally. Then the lesson's attack string in `innerHTML`:

```
const userInput = '';
box.innerHTML = userInput; // ❌ the alert fires
box.textContent = userInput; // ✅ harmless text
```

Stress: a `<script>` tag inserted with `innerHTML` does *not* run — it's the `onerror` attribute that does. That's why "it didn't run my script tag" is not proof of safety.

- 5 **classList (5m).** `add`, `remove`, `toggle`, `contains`. Then `card.className = "active"` and watch the `card` class vanish.
- 6 **Inline styles (4m).** camelCase (`backgroundColor`). Show `h1.style.fontSize` → `""`, then `getComputedStyle(h1).fontSize` → `"32px"`. Land the rule: *prefer classes over inline styles*.
- 7 **Attributes & dataset (4m).** `getAttribute("href")` vs `link.href` (resolved URL). `data-item-id` → `dataset.itemId`; values are always strings.
- 8 **Lab (8m).** Profile Card Editor.
- 9 **Check (2m).** Quiz Q1–Q5.
- 10 **Journal (3m).** End of Day 3 — give the full time.

◆ LIVE-DEMO SCRIPT

- 1 Open `labs/module-4/l14-profile-card/starter/`. In `script.js`, do task 6 (`card.style.backgroundColor = "#eff6ff"`), save, then click **Toggle dark mode**. The text goes light but the card stays light blue — unreadable.
- 2 Ask "why?" Inspect the card: the `style=""` attribute beats the `.dark` class. Add `card.style.backgroundColor = ""` before the toggle, save, click again. Fixed.
- 3 Say it: "Inline styles win the cascade. Toggle classes for looks; use `style` only for truly dynamic values."

◆ LAB — DYNAMIC PROFILE CARD EDITOR

The lesson's `profile.html`, split into `index.html` + `style.css` + `script.js`. Students complete TODOs 1–9 and the `updateProfile()` bonus. Starter in `labs/module-4/l14-profile-card/starter/` (button listeners provided — events are next lesson); solution in `labs/module-4/l14-profile-card/solution/`.

Circulate for: `dataset.user-id` (a subtraction, not a property!); setting `href` but not the link text (task 4 needs both); the bonus using destructuring `{ name, bio, location, website }` in the parameter list.

◆ WATCH-OUTS

- ◆ `innerHTML` with user text = XSS. Demo with `<img src=x onerror=...>`, never with `<script>` (it won't run and students conclude "it's safe"). "User input" includes form fields, URL parameters, API data, `localStorage`.
- ◆ `innerHTML +=` rebuilds everything inside the container — listeners are wiped and typed input values are lost. L16 shows the better way.
- ◆ `element.style` reads only inline styles. Reading `.style.color` to "check the current color" returns `""` for anything set in the stylesheet. Use `getComputedStyle`.
- ◆ Static `NodeList` vs live `HTMLCollection`. `querySelectorAll` is a snapshot; `children` updates itself. Re-query after adding elements.
- ◆ `null` from a wrong selector. Missing `#` or `.` is the usual cause; the error appears one line *later*, where the property is read.

◆ CHECK FOR UNDERSTANDING

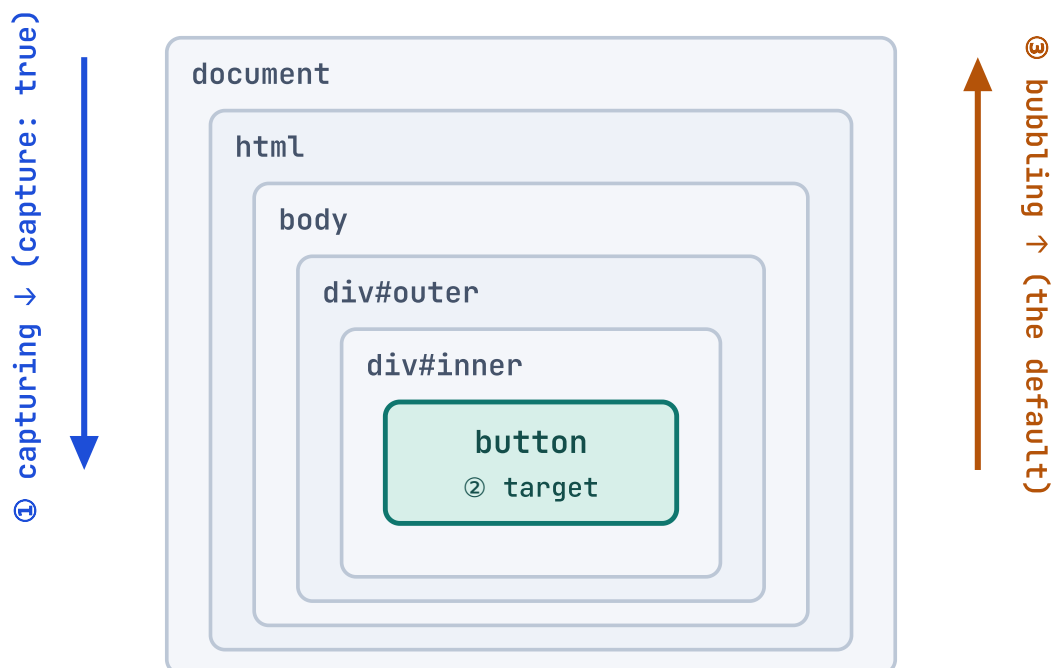
Key answers: no match → C — `null`; avoid `innerHTML` with user input because B — **it can execute malicious scripts (XSS)**; add a class safely = A — `classList.add`; `data-item-id` → B — `dataset.itemId`; stylesheet-only font size → `style.fontSize` is D — `""` (empty string).

◆ OBJECTIVES

- ◆ Attach listeners with `addEventListener` for click, input, submit and keydown
- ◆ Use the event object: `target`, `currentTarget`, `key`
- ◆ Cancel default behavior with `preventDefault()`; explain capturing and bubbling
- ◆ Delegate events from a parent with `closest()`; remove listeners correctly

◆ TEACHING SEQUENCE

- 1 **Frame (3m)**. Open Day 4 with a 60-second recap of selecting and modifying. Then the doorbell analogy: *the event is someone pressing the bell; the listener is what you decide to do when it rings*.
- 2 **addEventListener (5m)**. One button, one listener. Then the classic bug live: `addEventListener("click", handleClick())` — it runs once on load and never again. "Pass the function; don't call it."
- 3 **Event types & the event object (7m)**. `input` on a text box (every keystroke) vs `change` (on commit); `keydown` logging `event.key`. Log `event.target` and `event.currentTarget` on a card containing a button.
- 4 **preventDefault (5m)**. A form without it: the page reloads and the console clears. Add `event.preventDefault()` — it stays. Tick the Console's "Preserve log" to show the reload happening.
- 5 **Propagation (8m)**. The lesson's outer/inner/button example. Ask the room to *predict* the log order before clicking (that's also the journal prompt). Then add a capture listener on `#outer` and watch it jump to first. Use the diagram below.
- 6 **Delegation (8m)**. One listener on the list, `event.target.closest(".delete-btn")`. Add a new button after the fact — it just works. Then show why `matches()` fails when the click lands on an icon `<span>` inside the button.
- 7 **Removing listeners (4m)**. Named function in, same named function out. `{ once: true }`. Mention `AbortController` as "good to know."
- 8 **Lab (10m)**. Interactive Task List.
- 9 **Check (2m)**. Quiz Q1–Q6.
- 10 **Journal (3m)**.



A click travels down (capturing), reaches the target, then bubbles back up. Listeners run on the way up unless you pass `{ capture: true }`.

◆ LIVE-DEMO SCRIPT

- 1 Paste the lesson's three nested listeners into a blank page. "Before I click — what prints first?" Take three guesses, then click: *Button*, *Inner div*, *Outer div*.
- 2 Add `event.stopPropagation()` in the button listener, click: only "Button." Remove it again and say why: "It breaks delegation. Prefer checking `event.target`."
- 3 Open `labs/module-4/l15-task-list/solution/`, add a task, then type `<img src=x onerror=alert(1)>` as a task. It shows as plain text: "That's why we use `textContent` for anything a user typed."

◆ LAB — INTERACTIVE TASK LIST

The lesson's `tasks.html` as `index.html` + `style.css` + `script.js`. Students handle `submit` with `preventDefault()`, build each `<li>` (fixed markup via `innerHTML`, the user's text via `textContent`), delegate complete/delete with `closest()`, and write `updateCounter()`. Bonus: Escape clears the input; "d" toggles `body.dark-mode`. Files in `labs/module-4/l15-task-list/`.

Circulate for: listeners attached to each new button inside the submit handler (works, but misses the point — steer to delegation); counters that don't update after delete; the emoji buttons keeping their `aria-label`s ("Mark done", "Delete task") so screen readers announce them.

◆ WATCH-OUTS

- ◆ **Calling instead of passing:** `handleClick()` vs `handleClick`. The most common event bug of the day.
- ◆ `removeEventListener` needs the same function reference. A new arrow function that looks identical is a different object, so nothing is removed and there's no error.
- ◆ **focus, blur, mouseenter, mouseleave don't bubble**, so delegating them from a parent silently does nothing. Use `focusin` / `focusout` or `mouseover` / `mouseout`.
- ◆ **target** vs **currentTarget**. With delegation, `target` may be a child (an icon, a `<span>`). That's why `closest()` beats `matches()`.
- ◆ **Document-level shortcuts fire while typing.** The "d" shortcut must check that the input isn't focused, or every "d" in a task name toggles dark mode.

◆ CHECK FOR UNDERSTANDING

Key answers: `addEventListener` takes **B** — an event type string and a callback; `event.target` = **B** — the element actually interacted with; `preventDefault()` **C** — cancels the browser's built-in behavior; delegation's advantage = **C** — one listener for current and future children; fire once = **B** — `addEventListener("click", handler, { once: true })`; icon-span fix = **A** — `event.target.closest(".delete-btn")`.

◆ OBJECTIVES

- ◆ Create and configure elements with `createElement` before inserting them
- ◆ Insert with `append`, `prepend`, `before`, `after`; remove with `remove()`, `replaceWith()`, `replaceChildren()`
- ◆ Copy elements with `cloneNode(true)` and the `<template>` element
- ◆ Choose between `innerHTML` and `createElement`; batch inserts with a `DocumentFragment`

◆ TEACHING SEQUENCE

- 1 **Frame (3m)**. "So far we changed elements that were already there. Now we build new ones." Show the lesson's four-step flow: *create* → *configure* → *add children* → *insert*.
- 2 **createElement (6m)**. Create a `<p>` in the console, log it, point out it's *not on the page*. Configure it (text, class, `dataset`), then `document.body.append(p)` — now it appears.
- 3 **Inserting (7m)**. On a numbered list: `append`, `prepend`, `before`, `after`. Then append an *existing* item somewhere else — it *moves*. "One element, one place."
- 4 **Removing & clearing (5m)**. `el.remove()`, `replaceWith()`, and `list.replaceChildren()` to empty a container. Mention `removeChild` only as "what you'll see in old tutorials."
- 5 **Cloning (5m)**. `cloneNode(false)` vs `cloneNode(true)`; a quick `<template>` + `template.content.cloneNode(true)`. Note: clones don't copy `addEventListener` listeners — another reason to delegate.
- 6 **innerHTML vs createElement (5m)**. Walk the lesson's comparison table. Demo the `innerHTML +=` trap: type into an input inside a container, run `container.innerHTML += "<p>hi</p>"` — the typed text is gone.
- 7 **DocumentFragment (4m)**. Build 100 items in a fragment, append once, then log `fragment.childNodes.length` → 0. "The fragment is a shopping bag; you empty it onto the page."
- 8 **Lab (10m)**. Notification System.
- 9 **Check (2m)**. Quiz Q1–Q5.
- 10 **Journal (3m)**. This prompt also asks them to look back over the whole module.

◆ LIVE-DEMO SCRIPT

- 1 In the console: `const p = document.createElement("p"); p → <p></p>`. "Where is it on the page?" Nowhere. `p.textContent = "Hi!"; document.body.append(p)`.
- 2 Run `document.body.append(p)` a second time. "Did we get two?" No — it moved to the same spot. `document.body.append(p.cloneNode(true))` — now two.
- 3 Open `labs/module-4/l16-notifications/solution/`, click a few buttons, press **b** for the batch, then Clear All. Point at `aria-live="polite"` on the area: "screen readers announce each new toast."

◆ LAB — DYNAMIC NOTIFICATION SYSTEM

The lesson's `notifications.html` as `index.html` + `style.css` + `script.js`. Students write `showNotification(type, message)` with `createElement` + `textContent`, delegate the controls and the close buttons with `closest()`, clear with `replaceChildren()`, auto-remove after 5 seconds, and count with `updateCount()`. Bonus: the "b" key adds five at once through a `DocumentFragment`. Files in `labs/module-4/l16-notifications/`.

Circulate for: the close button missing `aria-label="Dismiss notification"`; the count not updating after the auto-remove timer; students trying Ctrl + Shift + N for the bonus — Chrome reserves it (new incognito window) and the page never sees the key. That's why the lesson uses "b".

◆ WATCH-OUTS

- ◆ "I created it but I can't see it." They forgot to insert it. `createElement` only builds it in memory.
- ◆ Appending moves, it doesn't copy. Looping `list.append(sameItem)` five times gives one item. Create a new element (or clone) each time.

- ◆ **Building with `innerHTML +=` in a loop** — slow, wipes listeners, and an XSS hole if the text came from a user. `createElement` + `textContent` + `append` instead.

- ◆ **The fragment is empty after insertion.** Students who try to reuse it get nothing the second time. Build a new fragment per batch.

- ◆ **CHECK FOR UNDERSTANDING**

Key answers: `createElement("div")` returns **B** — a new `<div>` in memory, not yet in the DOM; appending an existing element **B** — moves it; modern removal = **C** — `element.remove()`; `cloneNode(true)` vs `false` = **A** — deep vs shallow; `DocumentFragment` **C** — batches many insertions into one DOM update.

CLOSING THE MODULE

MODULE 4 WRAP-UP (10 MIN)

Have two or three students demo their task list or notification system on the projector, and ask each to point at one line that uses delegation and one that uses `textContent` for user text. Recap the arc: **see the tree → grab and change → respond to users → build and remove**. Preview Module 5: "You now have the full DOM toolkit. Next we put it to work on real features — forms with validation, timers, saving data, and live data from APIs."

Module 5 — Building Interactive Pages

How to teach the four lessons where students stop writing exercises and start building real features: a validated sign-up form, a Pomodoro timer, a to-do list that survives a refresh, and a live GitHub search. This guide covers timing, live-demo scripts, the async misconceptions to deal with early, and how to run each lab.

4 Lessons

~3.5 hrs teaching

Lessons L17–L20

Level Intermediate

Maps to Day 4 PM (L17–L18) → Day 5 AM

HOW TO USE THIS GUIDE

- Each lesson follows the same rhythm: **Frame** (why it matters), **Teach** (live-code the concept), **Lab** (students build), then **Check** (quiz and recap). The minute timings add up to the lesson's length. Adjust by ±10 min for your group. End each lesson with 3–5 minutes on the lesson's **Learning Journal** prompt, which appears in the lesson and in the workbook.
- Live-code everything**, with the **Console** and the relevant DevTools panel open: Elements for forms, Application for storage, Network for fetch. This module is all about things students can't see, like timers waiting in a queue, strings stored in the browser, and requests in flight. DevTools is how you show them.
- Blue blocks are concepts, **teal** blocks are hands-on, and **amber** blocks are watch-outs: the errors that eat class time if you don't head them off.
- The quiz answers here are for you. The student workbook has 3 questions from each lesson quiz, without answers, labeled with the lesson's question numbers. The separate **Answer Key** covers every question, with the same numbers, for grading.

BEFORE YOU START THE MODULE

Students need VS Code with **Live Server** running. L19's storage and L20's mock fallback both need an `http://` origin, not `file://`. They should be comfortable with `querySelector`, `addEventListener`, `createElement`, and `textContent` from Module 4. Each lab is a stand-alone folder in `labs/module-5/`. Hand out the **starter** folders at the start of the module. If someone falls behind, pair them up. Don't stop the whole room to fix one setup.

L20 NEEDS A PLAN: GITHUB'S RATE LIMIT

Without logging in, the GitHub API allows about **60 requests per hour per IP address**. A classroom behind one shared IP can use that up in minutes, and after that every request comes back **403**. Before Day 5, read the L20 lab notes below and `labs/module-5/README.md`. The solution has a clearly marked `USE MOCK` switch that uses `mock-user.json` instead of GitHub.

MODULE AT A GLANCE

LESSON	TOPIC	TIME	STUDENTS LEAVE ABLE TO...
L17	Forms & Validation	55m	Validate a form on blur and input, with accessible errors
L18	Timers & Animation	50m	Schedule, repeat, and cancel work; explain the event loop
L19	Local Storage	45m	Persist app state with JSON and a safe load/save pair
L20	Working with APIs & Fetch	60m	Fetch live data with async/await and handle every outcome

Unlike earlier modules, these are **four separate mini-apps**, one per lesson. They share one pattern that you should name out loud every time: **state → render → events → update state → render again**. L17 starts it (validation state drives what the page shows), L19 makes it explicit, and the L20 search app follows it too. By the end, students have four portfolio-ready pages and the tools for the Module 6 capstone.

L17 Forms & Validation

55 min

◆ OBJECTIVES

- ◆ Read form values with `.value` / `.checked` and through `form.elements`
- ◆ Handle `submit` with `event.preventDefault()`; collect data with `FormData`
- ◆ Write required, length, pattern (regex), and matching-field checks
- ◆ Validate on `blur`, then re-validate on `input` once a field is "touched"
- ◆ Show accessible errors (`role="alert"` , text as well as color) and know the Constraint Validation API
- ◆ Explain why the **server must validate again**

◆ TEACHING SEQUENCE

- 1 **Frame (4m).** Fill in a sign-up form with a bad email and hit Submit. The page reloads and everything is gone. Ask: "What would a *good* form have done?" Collect answers: warned me sooner, kept my data, told me what was wrong. That's today's list.
- 2 **Reading values (6m).** Type in an input, then read it in the console: `document.querySelector("#username").value` . Show that a number input still gives a *string*, and that a checkbox needs `.checked` , not `.value` . Then show `form.elements.email.value` as the tidy way to reach fields by `name` .
- 3 **Submit + preventDefault (7m).** Live-write the handler *without* `preventDefault()` first, so the console message flashes and disappears on reload. Then add it as the very first line:

```
form.addEventListener("submit", (event) => {
  event.preventDefault(); // first line, always
  const data = Object.fromEntries(new FormData(form));
  console.log(data);
});
```

Point out: listen for `submit` on the *form*, not `click` on the button. That way pressing Enter works too.

- 4 **Validation patterns (8m).** Build `isRequired` , `isLengthValid` , and the email regex `/^[^\s@]+@[^\s@]+\.[^\s@]+$/` . Read the regex aloud as "something, @, something, dot, something" and don't turn it into a regex lecture. Add the password-match check. Each validator returns `true` / `false` and shows or clears its own message.
- 5 **Real-time: blur then input (7m).** Show the "yelling form" first: validating on every keystroke from the start, so the email field turns red after one letter. Then add a `touched` Set: `blur` marks the field touched and validates it, and `input` only re-validates touched fields. Demo `showError` / `showSuccess` with `role="alert"` on the message span.
- 6 **Constraint API + the server rule (5m).** Quick tour: `required` / `minlength` / `pattern` , `input.validity` , `novalidate` , and `setCustomValidity("")` to clear. Then the most important part: disable JavaScript in DevTools (or point out that anyone can call the endpoint with `fetch()` from the console) and show that the garbage gets sent without any checks. *"Anything in the browser can be bypassed. Client-side validation is for user experience. The server always checks again."*
- 7 **Lab (15m).** Launch the registration-form lab (below).
- 8 **Check + journal (3m).** Quick quiz poll, then the journal prompt.

◆ LAB – REGISTRATION FORM WITH REAL-TIME VALIDATION

Students complete TODOs 1–5 in `script.js` : helpers, four validators, a 0–5 password strength meter, blur-then-input wiring, and a submit handler that hides the form and shows the success message. Starter in `labs/module-5/l17-forms-validation/starter/` ; solution in `labs/module-5/l17-forms-validation/solution/` . The lesson calls the page `registration.html` ; the lab splits it into `index.html` + `style.css` + `script.js` .

Run these three test cases on your screen and have every student run them too: (1) **empty submit**: all four errors appear and focus jumps to Username; (2) a **weak password** like `abc` : the meter reads "Very weak" and blur shows the length error; (3) type a **mismatched confirm**, tab out to see "Passwords

do not match," then go back and *change the password* to match. The confirm error should clear on its own. If it doesn't, they're missing the "password changed → re-check confirm" line.

◆ WATCH-OUTS

- ◆ **preventDefault() in the wrong place.** If a bug throws before it runs, the page reloads and wipes the console, so the error looks like "nothing happens." Put it on line one. Tick **Preserve log** in the console to catch it.
- ◆ **.value** is always a string. `"25" > "100"` is `true`. Convert with `Number()` before comparing.
- ◆ **Short-circuiting the validators.** `validateA() && validateB()` stops at the first failure, so the other errors never show. The solution uses `map()` to run them all, then `every(Boolean)`.
- ◆ **Duplicate error spans.** Creating a new span on every blur stacks messages. Find the existing one first, create only if it's missing, and set `textContent` (never `innerHTML`).
- ◆ **"My JS validation makes it secure."** It doesn't. Repeat the server rule until students can say it back.

◆ CHECK FOR UNDERSTANDING

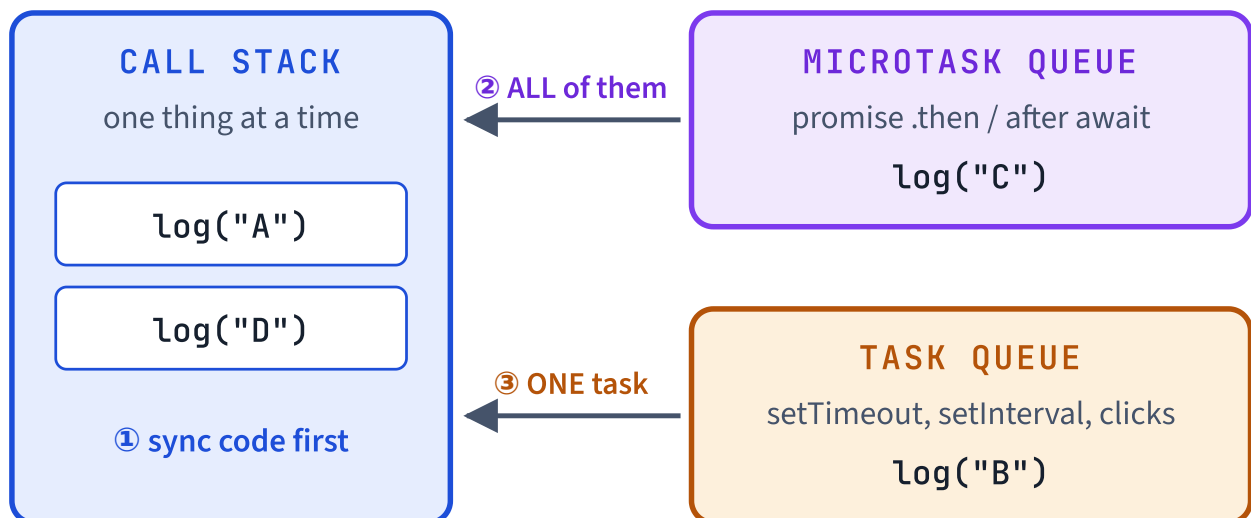
Key answers: read a text input with A) `input.value`; `preventDefault()` in submit C) **stops the reload/navigation**; checkbox state = D) `checkbox.checked`; `blur` is for B) **validating after the user leaves a field**; client-side isn't enough because C) **users can bypass it**; the stuck custom message = D) **never cleared with** `setCustomValidity("")`.

◆ OBJECTIVES

- ◆ Delay and cancel with `setTimeout` / `clearTimeout` ; repeat and stop with `setInterval` / `clearInterval`
- ◆ Build a countdown; know when recursive `setTimeout` beats `setInterval`
- ◆ Tell **debounce** from **throttle**
- ◆ Explain the event loop: call stack, microtask queue, task queue
- ◆ Animate with `requestAnimationFrame` or, better, by toggling a class for a CSS transition

◆ TEACHING SEQUENCE

- 1 **Frame (3m)**. Show the finished Pomodoro timer from `labs/module-5/l18-timers-animation/solution/` on the 1-minute preset. "By the end of the hour, you'll have built this."
- 2 **setTimeout + cancel (6m)**. `setTimeout(showWelcome, 2000)` , then the classic bug `setTimeout(showWelcome(), 2000)` : it fires *immediately*. "Parentheses call it now. No parentheses hands it over for later." Store the returned ID and cancel it with `clearTimeout` .
- 3 **setInterval + countdown (6m)**. Live-code `startCountdown(10, display)` and cancel it with `clearInterval` when it reaches 0. Draw the recursive- `setTimeout` comparison: the interval fires on a fixed rhythm, while the recursive timeout waits until each run is done.
- 4 **Debounce vs throttle (6m)**. Type into a search box with a plain listener and the console floods. Wrap it in `debounce(fn, 300)` and the messages come only after typing stops. Compare it to an elevator door that waits for people to stop coming in. Throttle is "at most once every N ms," which suits scroll. Tell them to keep `debounce` handy: it comes back in L20.
- 5 **The event loop (7m)**. Ask the room to *predict* the output before you run anything. Open `labs/module-5/l18-timers-animation/demo/event-loop.html` , click **Run it**, and get A, D, C, B. Walk through the diagram below: sync code runs first, then *all* microtasks (promises), then *one* task (timers). Then click the quiz button: **sync, promise, timer**.



Output: A · D · C · B

The event loop: sync code, then every microtask, then one task, and repeat.

Land the takeaway: *a timer's delay is a minimum, not a promise*. This is also why the stretch timer calculates from `Date.now()` instead of counting ticks.

- 6 **RAF + CSS transitions (5m).** Show one `requestAnimationFrame` loop using the timestamp, then the easier approach: CSS defines the transition and JS only toggles a class. Mention `prefers-reduced-motion`: big motion is something users should be able to opt out of.
- 7 **Lab (14m).** Launch the Pomodoro lab.
- 8 **Check + journal (3m).**

◆ LAB – POMODORO TIMER WITH ANIMATED PROGRESS

TODOs 1–8: DOM references, state, `updateDisplay()` (MM:SS with `padStart`, plus the ring's `stroke-dashoffset = CIRCUMFERENCE * (1 - remaining/total)`, where $r = 130$ and the circumference is **816.81**), start/pause/reset, the 25/15/5/1-min presets, and the bonus colors: **red under 30 s, green at 0**. Files in `Labs/module-5/l18-timers-animation/`: `starter/`, `solution/` (the lesson's tick-counting version), and `stretch/`.

Testing tip: nobody should wait 25 minutes. Use the 1-min preset, or type `remainingSeconds = 5` in the console before pressing Start. **Stretch for fast finishers:** the lesson's "Going further." Store `endTime = Date.now() + remainingSeconds * 1000` on Start and calculate `Math.ceil((endTime - Date.now()) / 1000)` on each tick. Then switch tabs for 30 s and compare the two versions. Also add a `prefers-reduced-motion` rule that turns off the ring transition. Reference: `stretch/`.

◆ WATCH-OUTS

- ◆ **Double-click Start = double speed.** Each click starts another interval, and the timer runs 2× fast and can't be paused. The `if (isRunning) return;` guard fixes it. Let a student find this bug. It teaches more than any slide.
- ◆ **Lost interval ID.** `const intervalId = setInterval(...)` declared *inside* `start()` can't be reached by `pause()`. State lives outside the functions.
- ◆ **Calling instead of passing:** `addEventListener("click", start())` runs `start` once, at load.
- ◆ **The ring goes the wrong way or doesn't move.** Check that `stroke-dasharray` is 816.81 and the formula is `1 - remaining/total`. The SVG is rotated -90° so the ring starts at 12 o'clock.

◆ CHECK FOR UNDERSTANDING

Key answers: `setTimeout` returns D) a numeric timer ID for `clearTimeout()`; A / timeout B / C logs B) A, C, B; interval vs recursive timeout = C) interval fires regardless; recursive waits for completion; rAF because A) it syncs with the display and pauses in background tabs; debouncing = D) delay until events stop, resetting on each event; timer/promise/sync logs B) sync, promise, timer.

◆ OBJECTIVES

- ◆ Use `setItem`, `getItem`, `removeItem`, `clear`
- ◆ Store arrays and objects with `JSON.stringify` / `JSON.parse`, with a safe fallback
- ◆ Choose between `localStorage` and `sessionStorage`
- ◆ Apply the load → render → update → save pattern; know the `storage` event
- ◆ Know the limits (~5 MB, synchronous) and never store secrets

◆ TEACHING SEQUENCE

- 1 **Frame (3m).** Add three tasks to a plain to-do list, then refresh the page, and they're gone. "Everything we've built so far forgets on refresh. Today it starts remembering."
- 2 **The four methods + DevTools (6m).** Split the screen: console on one side, **Application → Local Storage** on the other. `setItem("theme", "dark")` and watch the row appear. `getItem` on a missing key gives `null`. Then `setItem("score", 42)` and `typeof getItem("score")` gives `"string"`.
- 3 **The JSON round-trip (7m).** `setItem("user", {name: "Ray"})` stores `"[Object Object]"`. Show it in DevTools, because the reveal matters. Then show the fix: `JSON.stringify` on the way in, `JSON.parse` on the way out, and the `loadData(key, fallback)` helper with try/catch. Corrupt the value by hand in DevTools and reload to show why the catch matters.
- 4 **session vs local (4m).** Same API, different lifetime. Open a second tab: `localStorage` is shared across tabs and `sessionStorage` is not. Quick rule: "Would the user expect it tomorrow? Use local."
- 5 **State-sync pattern + storage event (5m).** Write the pattern on the board: **load once → render → on every change: update the array, save, render**. The array is the truth, and the page is drawn from it. Mention the `storage` event: it fires in the *other* tabs, never in the one that made the change.
- 6 **Limits & security (4m).** About 5 MB, synchronous, `QuotaExceededError`. Then the rule: *any script on the page can read storage, so never store passwords or tokens there*. HttpOnly cookies are the answer for auth.
- 7 **Lab (13m).** Launch the to-do lab.
- 8 **Check + journal (3m).**

◆ LAB – PERSISTENT TO-DO LIST

Seven TODOs: 1 `loadTodos` / `saveTodos` (try/catch, `[]` fallback) · 2 `render` (empty state; checkbox, `span.todo-text`, `button.delete-btn`) · 3 `addTodo` · 4 `toggleTodo` / `deleteTodo` · 5 `updateCount` · 6 listeners, with **event delegation** on the `<ul>` · 7 load and render on start. Files in `Labs/module-5/l19-local-storage/`.

Accessibility check: without an accessible name, a screen reader announces the checkbox and the × button as just "checkbox" and "times." The lesson and the lab solution both add `aria-label`s (`Mark "Buy milk" as done`, `Delete "Buy milk"`), a visually-hidden `<label>` for the input, and puts focus back after `render()` rebuilds the list. Show this at the end of the lab as the "production polish" step. **Success check:** add two tasks, tick one, refresh, and everything is still there.

◆ WATCH-OUTS

- ◆ **Forgetting to save.** The list works until refresh, then comes back stale. Every function that changes `todos` must call `saveTodos`.
- ◆ **ID type mismatch.** `li.dataset.id` is a `string` and `todo.id` is a `number`, so `===` never matches and toggle/delete silently do nothing. The fix is `Number(li.dataset.id)`.
- ◆ **Listeners on each item instead of delegation.** They vanish every time `render()` clears the list. One listener on the `<ul>` + `closest(".todo-item")` handles everything.
- ◆ **Shared origin in class.** Every student project on `127.0.0.1:5500` shares the same `todos` key. Old demo data can show up in a new project. Teach `localStorage.removeItem("todos")`.
- ◆ **innerHTML with task text.** Always use `textContent`. Have someone add the task `<b>hi</b>` to prove it shows as plain text.

◆ CHECK FOR UNDERSTANDING

Key answers: missing key returns B) `null` ; storing an object directly gives D) `"[object Object]"` ; store an array with A)

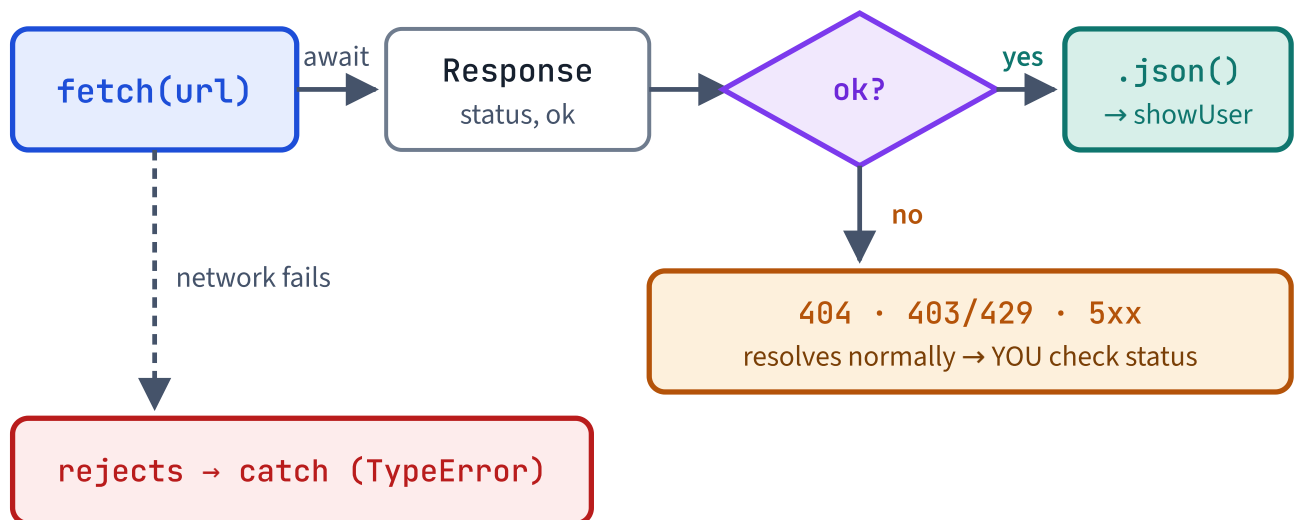
`JSON.stringify(myArray)` ; sessionStorage C) **clears when the tab closes**; no tokens because C) **any JS on the page can read it (XSS)**; the `storage` event fires B) **in the other tabs, not Tab A**.

◆ OBJECTIVES

- ◆ Explain APIs, endpoints, REST methods, and JSON
- ◆ Describe a Promise's three states; chain `.then()` / `.catch()`
- ◆ Use `fetch()`, check `response.ok`, parse with `response.json()`
- ◆ Rewrite with `async` / `await` + try/catch; run requests in parallel with `Promise.all`
- ◆ Show loading, data, and error states safely (escape or `textContent`)

◆ TEACHING SEQUENCE

- 1 **Frame (4m)**. Paste `https://api.github.com/users/octocat` into the browser address bar, and raw JSON appears. "That's an API. Today your page asks for this and turns it into a profile card."
- 2 **APIs, REST, JSON (5m)**. Request → endpoint → response. Keep the GET/POST/PUT/DELETE table short: today is almost all GET.
- 3 **Promises (8m)**. Use the restaurant-buzzer analogy: you get the buzzer (the Promise) right away and the food (the value) later. Three states: pending, fulfilled, rejected. Live-code the `.then().then().catch().finally()` chain against JSONPlaceholder, then remove a `return` to show the next `.then` receiving `undefined`.
- 4 **fetch + response.ok (8m)**. Fetch `/users/9999` from JSONPlaceholder. It's a 404, but the `.then` still runs. This is the big surprise: **fetch only rejects for network failures**. Add `if (!response.ok) throw ...`. Open the **Network** tab and show the request, its status, and its response body. Use the diagram below.



Only a network failure lands in catch. HTTP errors resolve, so check response.ok.

- 5 **async/await + try/catch (8m)**. Rewrite the same request with `async` / `await`, side by side with the `.then` version. Point out that `await` only works inside an `async` function (or a module). Show `Promise.all`: the total time is the slowest request, not the sum of all of them.
- 6 **Displaying data (6m)**. Loading → data | error. Render with `createElement` + `textContent`, and show `escapeHTML` for when you must use a template string. Briefly mention CORS: the fix is on the server, not in your code.
- 7 **Lab (18m)**. Launch GitHub User Search, using the rate-limit plan below.
- 8 **Check + journal (3m)**.

◆ LAB – GITHUB USER SEARCH

TODOs 1–7: DOM references; `showLoading` / `showError` / `showUser` (avatar, name, @login, bio, repos/followers/following, all as text); `async searchUser` with separate branches for 404, 403/429 (rate limit), other statuses, and a `catch` for network errors; click + Enter; bonus debounce and

profile link. Files in `labs/module-5/l20-apis-fetch/`.

Rate-limit plan (60 requests/hour per IP, and the room shares one IP): (1) **pair up**. One laptop searches while the partner reads the code and predicts the branch. (2) **No auto-search in class**: debounced search-as-you-type spends a request on every pause. It's off (`AUTO_SEARCH = false`) in the solution, so demo it once and turn it off again. (3) Test **404** with one made-up name rather than many. (4) When the room hits 403, switch to the **fallback**: open the solution as `index.html?mock` or set `USE MOCK = true`. In mock mode, `octocat` returns the sample profile from `mock-user.json`, `ratelimit` simulates a 403, and any other name simulates a 404. Students can point their own fetch at `mock-user.json`; the starter's top comment explains how. Treat the 403 as a teaching moment: this is the error branch they just wrote, working.

WATCH-OUTS

- ◆ **Forgetting `await`**. `const user = response.json()` logs `Promise {<pending>}` and every field is `undefined`. When you see "Promise" in the console, look for a missing `await`.
- ◆ **Expecting 404 in catch**. It never gets there. Branch on `response.status` / `response.ok` *before* parsing.
- ◆ **Raw API text in `innerHTML`**. Bios are user-written. Use `textContent`, or `escapeHTML` when you use a template.
- ◆ **Double requests**. Enter + a debounced input listener both fire, which burns quota twice as fast.
- ◆ **file:// URLs**. Some browsers block or restrict requests from `file://` pages, and the mock JSON won't load. Use Live Server.

DISCUSSION (5 MIN, IF TIME ALLOWS): THE STALE-RESPONSE RACE

With debounced search, a student types `oc`, pauses, then finishes `octocat`. Two requests are now in flight. If the `oc` answer comes back *last*, it overwrites the right result. Ask: "The responses came back in a different order than we sent them. How do we make sure only the newest search wins?" Take ideas, then show the solution's guard: a `latestSearch` counter, where each search remembers its own number and returns early after every `await` if a newer search has started. (`AbortController` is the more advanced tool. Mention it by name only.)

CHECK FOR UNDERSTANDING

Key answers: `fetch()` returns B) a Promise that resolves to a Response; on 404 A) the Promise resolves with `ok = false`; `async` C) always returns a Promise and enables `await`; `Promise.all` because B) parallel: total = slowest, not the sum; the three UI states = D) loading, data, error; the CORS error = C) the server didn't allow your origin.

CLOSING THE MODULE

MODULE 5 WRAP-UP (10 MIN)

Ask for one volunteer per app to demo: the form catching a bad email, the timer turning red, the to-do list surviving a refresh, and the search finding a real profile (or the mock one). Recap the arc: **capture input → act over time → remember → talk to the world**. Then name the pattern they used four times: state → render → events → update. Preview Module 6: "You now have every tool you need. Next you put them together into a capstone app of your own."

Module 6 — Capstone & Next Steps

The finale. Students build QuickNote, a complete note-taking app that uses every skill from the course, then learn to debug like a professional and plan what to build next. End the course with a working app on every screen and a clear next step in every workbook.

2 Lessons

~2.3 hrs teaching

Lessons L21-L22

Level Intermediate

Maps to Day 5 PM (capstone)

HOW TO USE THIS GUIDE

- This is the **capstone**. L21 is one 90-minute **guided build**: you code each step live, the room mirrors you, and everyone passes a checkpoint before you move on. No step is "go build it yourself."
- **Live-code everything, and let it break**. A missing comma or a typo'd ID in front of the class is a free debugging lesson. Read the red error out loud, find the line, fix it. That rehearses L22 before L22 starts.
- **Nobody falls off the train**. Each step has a catch-up file in `labs/module-6/l21-quicknote/checkpoints/`. If a student is more than a step behind, hand them the checkpoint and let them rejoin at the next TODO. Finishing matters more than typing every line.
- Blue blocks are concepts; **teal** blocks are hands-on; **amber** blocks are watch-outs.
- Quiz answers here are for you. The student workbook has a 3-question subset of each lesson quiz, without answers, labelled with the lesson's question numbers; the separate **Answer Key** covers every question with matching numbers for grading.
- L22 closes the whole course. Protect the last 15 minutes for the final **Learning Journal** entry and the course wrap-up. Don't let a stubborn bug eat the goodbye.

PRE-FLIGHT FOR THE CAPSTONE

QuickNote fetches a quote from `https://dummyjson.com/quotes/random`. Open it in the classroom browser before class. If the network blocks it, that's fine: the app shows its fallback quote, and you get to teach graceful degradation with a real outage. Have Chrome or Edge with DevTools on every machine, Live Server working, and the starter plus the four checkpoint files ready to share (a USB stick or a shared folder). Test `labs/module-6/l22-debugging/starter/` once through Live Server so you've seen all seven bugs yourself.

BEFORE YOU START THE MODULE

Students need the Module 5 skills fresh: form validation (L17), timers and debounce (L18), `localStorage` (L19), and `fetch` with `async/await` (L20). Spend two minutes after lunch asking "what does `JSON.stringify` do? what does `await` wait for?" If anyone's setup is broken, pair them now. The capstone is no place for a first Live Server install.

MODULE AT A GLANCE

LESSON	TOPIC	TIME	STUDENTS LEAVE ABLE TO...
L21	Building a Complete Project	90m	Plan and build a complete CRUD app with storage, search, a fetched quote and shortcuts
L22	Next Steps & Best Practices	45m	Debug systematically, organize code, avoid common pitfalls, and choose what to learn next

Suggested afternoon: L21 (90m) → break (10m) → L22 (45m) → course wrap-up (15m). If the build runs long, trim L22's ecosystem tour (step 6), never the bug hunt or the final journal.

L21 Building a Complete Project

OBJECTIVES

- Plan an app before coding: feature list, data model, architecture
- Separate a **data layer** (CRUD + `localStorage`) from **rendering** and **event handling**
- Use one form for create *and* edit, with validation and a timed status message
- Add debounced search, a category filter, a fetched quote with a fallback, event delegation and keyboard shortcuts
- Keep user text safe with `textContent`

TEACHING SEQUENCE

- Frame: show the finished app (5m).** Open `solution/quicknote.html`. Add a note, edit it, search, filter, reload ("it's still there"), press N. Then say: "Every piece of this is something you already learned. Today we assemble it."
- Plan and architecture (7m).** Walk the feature table (feature → skill → lesson), the note object (`id` from `Date.now()`, ISO timestamps), and the loop from the workbook diagram: **user action** → **update state** → **save** → **re-render**. Write the loop on the board and leave it up all afternoon.
- Step 1: tour the starter (3m).** Everyone opens `starter/quicknote.html` with Live Server. Point out the hidden form, the empty `#notes-grid`, the `role="status"` span, and the DOM references already written at the top of the script.
- Step 2: data layer (10m).** Live-code `loadNotes` / `saveNotes` with try/catch, then the CRUD functions and `getNotes`. Stress: *no DOM in here*. **Checkpoint:** `createNote("Test", "Hello", "work")` in the Console, then `notes`, then Application → Local Storage.
- Step 3: rendering (12m).** `formatDate`, then `renderNotes`: clear, empty state, build cards in a `DocumentFragment`, append once, update the count. Uncomment `renderNotes()` in INIT. **Checkpoint:** reload and the Console-made note is a card. (Catch-up: `step-3-render.html`.)
- Step 4: form and validation (15m).** `showForm` / `hideForm`, field errors placed right after their input, `validateForm`, `showSaveStatus` with `clearTimeout`, the submit handler. Land "one form, two modes": the hidden `#note-id` decides create vs update. **Checkpoint:** saving empty shows both messages; a valid save shows "Note saved!". (Catch-up: `step-4-form.html`.)
- Step 5: search and filter (7m).** Type `debounce` from memory with the room (L18 callback). Wire `input` → debounced `applyFilters`, `change` → `applyFilters`. Point out the payoff: `renderNotes` already took the parameters. (Catch-up: `step-5-search.html`.)
- Step 6: the quote (6m).** `fetchQuote` with `res.ok` check and a fallback in `catch`. **Checkpoint:** DevTools → Network → Offline, reload: fallback quote, app still works. (Catch-up: `step-6-quote.html`.)
- Step 7: polish (12m).** One delegated click listener on `#notes-grid` for  and  (including closing the form if you delete the note being edited), then the keyboard shortcuts. The transition CSS is already in the starter. Explain why the code uses N and not Ctrl+N.
- Test pass (5m).** Students run the checklist in the lab README (or the workbook) against their own app and fix what fails.
- Quiz + journal (8m).** Run Q1–Q5 as a poll (3m), then 5 minutes on the journal prompt.

LIVE-DEMO SCRIPT – THE THREE MOMENTS TO REHEARSE

- The loop, visible.** After Step 3, run this in the Console and narrate each line as a link in the chain:

```
createNote("Buy milk", "2 liters", "personal"); // update state + save
renderNotes();                                // re-render
location.reload();                             // load → still there
```

- The `textContent` test.** Once the form works, save a note titled `<img src=x onerror=alert(1)>`. It shows as text. Then, in the Console, briefly swap one line to `title.innerHTML = note.title`, re-render, and let the alert fire. Undo it at once. Nobody forgets that demo.
- Delete while editing.** Open a note in the form with , then delete that same note. Show why the delegation code calls `hideForm()` first:

```
if (noteIdInput.value === String(noteId)) hideForm();
```

Without it, "Update Note" would try to update a note that no longer exists.

◆ LAB - BUILD QUICKNOTE (GUIDED)

Everyone starts from `labs/module-6/l21-quicknote/starter/quicknote.html` (HTML + CSS done; TODOs 2a–7b in the script). Catch-up files for Steps 3–6 are in `checkpoints/`; the finished app is `solution/quicknote.html` (the lesson's complete source). The lab README has the step table, the test checklist, and the 8 stretch challenges as extensions.

Facilitator test checklist (walk the room with it): add ☐ · empty save shows *both* messages ☐ · edit ("Update Note") ☐ · delete with confirm and fade ☐ · delete the note being edited closes the form ☐ · search is debounced ☐ · category filter ☐ · reload keeps notes ☐ · quote loads and the offline fallback works ☐ · N / / Esc work, and typing "n" in a field doesn't open a note ☐ · an HTML-looking title shows as plain text ☐ · clean Console ☐.

Early finishers: the extensions list (dark mode, export/import, drag & drop, Markdown preview, multi-tab sync, word count, confirmation modal, pin notes). Word count and pin notes fit in 10 minutes.

◆ WATCH-OUTS

- ◆ **Using a function before its step exists.** `ReferenceError: renderNotes is not defined` usually means they uncommented INIT early. The starter keeps INIT commented for this reason.
- ◆ **IDs as strings.** `card.dataset.id` and `#note-id` hold strings, but note ids are numbers. `notes.find(n => n.id === "1713...")` finds nothing. Hence `Number(...)` in the lesson code.
- ◆ **Field errors that pile up or jump.** If `clearFormErrors()` isn't called at the start of `validateForm()`, each failed save adds another message. If the error span is searched in the whole parent, the title's message can land under the body.
- ◆ **Listeners inside `renderNotes()`.** Some students attach click handlers to each card's buttons. They work until the next render. This is exactly why the lesson uses one delegated listener (quiz Q4).
- ◆ **"N typed into the title."** Forgetting `event.preventDefault()` puts an "n" in the freshly focused title; forgetting the `closest("input, textarea, select")` guard opens a new note whenever someone types "n".
- ◆ **Ctrl+N.** Students will ask. Chrome reserves it for a new window and never passes it to the page, so `preventDefault()` can't help. Single-key shortcuts outside fields are the Gmail and GitHub convention.
- ◆ **Stale data from earlier attempts.** A half-built `createNote` can save malformed notes that break rendering later. Fix: Application → Local Storage → delete `quicknote_notes`, reload.

◆ CHECK FOR UNDERSTANDING

Key answers: one form knows create vs edit because it **checks whether the hidden `note-id` input has a value** (Q1 · B); `textContent` so user text is **shown as plain text, never run** (Q2 · C); `debounce(applyFilters, 300)` = **re-render only after typing pauses for 300ms** (Q3 · A); one listener on `#notes-grid` because **cards are rebuilt every render — event delegation** (Q4 · D); quote API down = **fallback quote, app keeps working** (Q5 · B).

Journal (5m): students trace one feature (deleting a note, say) through *user action* → *update state* → *save* → *re-render*, naming the functions it calls, and note one feature they'd add and one part of the flow they're unsure about.

◆ OBJECTIVES

- ◆ Read an error message for **what**, **where** and a likely **why**; use the debugging loop instead of random changes
- ◆ Use `console.table` / `group` / `time` / `assert`, `debugger` and breakpoints
- ◆ Organize code by concern, name things clearly, keep functions small, split files with modules
- ◆ Recognize the common pitfalls: `null` queries, the async trap, `==`, shallow copies, duplicate listeners, edge cases
- ◆ Read modern syntax (`?.`, `??`, destructuring, spread/rest) and pick a next step: projects, Git, npm + Vite, a framework, TypeScript, testing

◆ TEACHING SEQUENCE

- 1 **Read the error (5m)**. Project `Cannot read properties of null (reading 'textContent')` and translate it together: WHAT (reading on `null`), WHERE (file:line, function), WHY (the selector matched nothing). Walk the five-row error table; stress *"Unexpected token → check the line before."*
- 2 **Console power tools + the debugger (7m)**. Live: `console.table(notes)` on QuickNote, a `console.group`, a `console.time` around a fetch. Then drop `debugger;` into `deleteNote`, reload with *DevTools open*, step over, hover variables. Finish with a Sources-panel breakpoint (no code change).
- 3 **The debugging mindset (3m)**. Use the workbook loop: read → reproduce → hypothesis → test → fix → verify. Mention rubber-duck debugging and mean it.
- 4 **Bug-hunt lab (15m)**. Launch `Labs/module-6/L22-debugging/`. Students fill the worksheet row by row. At minute 12, debrief bugs 6 and 7 together if the room is stuck.
- 5 **Code organization + pitfalls (5m)**. Use QuickNote as the example: data / rendering / events sections, naming conventions, "if it's over 20–30 lines, split it," and the module file structure (`type="module"` needs a server). Tick off the pitfalls the lab already covered; spend your words on shallow copy vs `structuredClone` and `==` vs `===`.
- 6 **Modern features, ecosystem, learning path (5m)**. One slide each: `?.` and `??` (0 is kept), destructuring, spread/rest. Then the map: frameworks (React/Vue/Svelte), Vite + npm (`npm create vite@latest`, not the deprecated `create-react-app`), Node LTS, TypeScript, testing. Close on the suggested order and the 80/20 rule: *build more than you watch*.
- 7 **Quiz + final journal (5m)**. Q1–Q5 as a quick poll (2m); then the final journal prompt (3m in class, finish at home). Point students at the follow-on project menu in the workbook.

◆ LIVE-DEMO SCRIPT – THE ASYNC TRAP IN 30 SECONDS

```
let data;
fetch("https://dummyjson.com/quotes/random")
  .then(r => r.json()).then(d => { data = d; });
console.log(data); // undefined – fetch hasn't finished
setTimeout(() => console.log(data), 1000); // now it's there
```

Run it in the Console, then rewrite it with `await`. This is Bug 7 in the lab, so it lands twice.

◆ LAB – BUG HUNT: SNACK BUDGET

Students open `Labs/module-6/L22-debugging/starter/` with Live Server and fix **seven planted bugs** in `script.js`, logging each in the worksheet (`L22-debugging/README.md`, also in the workbook). Five crash, in this order: missing comma (`Unexpected token '{'`) → missing settings (`reading 'weekly'` of undefined) → selector typo (`null`) → `formatprice is not defined` → `items.length is not a function`. Two are silent: **Bought** works once, then never again (a listener added inside `render()`), and **Load sample items** uses `samples` before `fetch` finishes. Fixed version with `// FIX n` comments: `L22-debugging/solution/`.

Circulate for: students changing code before reading the message; fixing the reported line of the syntax error instead of the line before; "fixing" bug 6 by adding `removeEventListener` gymnastics instead of moving the listener out; wrapping bug 7 in a `setTimeout` "because it works now." That last one is a race, not a fix.

◆ WATCH-OUTS

- ◆ **Panic at red text.** Model calm: read it aloud, click the file:line link. The message is a clue, not a verdict.
- ◆ **"The debugger does nothing."** DevTools must be open *before* the reload; otherwise `debugger` is ignored (quiz Q5).
- ◆ `||` vs `??` — `score || 10` turns a real 0 into 10. Use `??` for defaults when 0, `""` or `false` are valid.
- ◆ **Spread is shallow.** `{ ...original }` shares nested arrays; `structuredClone` makes a deep copy.
- ◆ **Ecosystem overwhelm.** Some students leave thinking they must learn everything at once. Point at the suggested order: more vanilla projects first, then Git. Frameworks make more sense once they've built without one.
- ◆ **Outdated tutorials.** Warn that `create-react-app` still appears in search results; it was deprecated in 2025. Start with Vite or a meta-framework.

◆ CHECK FOR UNDERSTANDING

Key answers: `Cannot read properties of null (reading 'textContent')` = `querySelector` found nothing and returned `null` (Q1 · C); `player.score ?? 10` with a score of 0 = 10 (Q2 · A); pushing onto `copy.scores` after a spread copy changes `original.scores` too — shared nested array (Q3 · B); a new React project today = Vite or a framework like Next.js (Q4 · C); `debugger` didn't pause because DevTools wasn't open (Q5 · B).

Final journal: which skill felt impossible at the start and makes sense now; one project from the Project Ideas table (or their own), the lessons it would use, and the very first step; one topic to revisit.

CLOSING THE COURSE

COURSE WRAP-UP (15 MIN) 🎉

Showcase: two or three students demo their QuickNote on the projector, including one extension if anyone got that far. Recap the whole journey, one line per module: **foundations → control flow & functions → data → the DOM → interactive pages → a complete app**. Have everyone circle one project from the follow-on menu in the workbook and write its first step. Suggest they deploy QuickNote the way they deployed their HTML & CSS site, since a live link belongs in a portfolio. Hand out completion recognition. Remind them that every professional started exactly here: the difference is more time and more projects.